



**UNIVERSIDAD
DE GRANADA**

**TRABAJO FIN DE GRADO
INGENIERÍA INFORMÁTICA**

Aplicación móvil para la detección precoz de melanomas utilizando técnicas de aprendizaje profundo

Autor

José Illana Lope

Directores

Carlos Gustavo Porcel Gallego



**ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN**

—
Granada, septiembre de 2025

Aplicación móvil para la detección precoz de melanomas utilizando técnicas de aprendizaje profundo

José Illana Lope

Palabras clave: Redes neuronales, Inteligencia artificial, Melanomas, Vision por computador, Redes neuronales convolucionales

Resumen

En los últimos años se ha producido un avance sin precedentes en el campo de la inteligencia artificial. Este avance se ha podido observar en diversos campos, tales como la industria, con la automatización de procesos que antes requerían supervisión humana, la educación, con la aparición de herramientas que se apoyan en la inteligencia artificial para potenciar el aprendizaje y la adquisición de nuevos conceptos, o la medicina.

En este último campo cabe destacar el análisis de imágenes. Haciendo uso de la inteligencia artificial se han podido analizar miles de imágenes médicas en busca de patrones con el objetivo de encontrar patrones en ellas que ayuden a detectar y diagnosticar enfermedades de forma precoz.

El objetivo principal de este proyecto ha sido el de combinar los avances que se han producido en este campo y acercarlos a los usuarios por medio del desarrollo de una aplicación móvil. Esta aplicación analizará imágenes tomadas por el usuario de sus lunares y tratará de determinar la probabilidad de que estos puedan ser cancerígenos.

Mobile application for early detection of melanomas using deep learning techniques

José Illana Lope

Keywords: Neural networks, Artificial intelligence, Melanoma, Computer Vision, Convolutional neural networks

Abstract

In recent years, there has been unprecedented progress in the field of artificial intelligence. This progress has been observed in various fields, such as industry, with the automation of processes that previously required human supervision; education, with the emergence of tools that rely on artificial intelligence to enhance learning and the acquisition of new concepts; and medicine. In this last field, image analysis is particularly noteworthy. Using artificial intelligence, thousands of medical images have been analyzed in search of patterns that can help detect and diagnose diseases at an early stage. The main objective of this project has been to combine the advances that have been made in this field and bring them closer to the average user through the development of a mobile application. This application will analyze images taken by the user of their moles and try to determine the probability of them being cancerous.

Yo, **José Illana Lope**, alumno de la titulación Grado en Ingeniería Informática de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 26517462A, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: José Illana Lope

Granada a 5 de septiembre de 2025.

D. **Carlos Gustavo Porcel Gallego (tutor)**, Profesor del Departamento de Ciencias de la Computación e Inteligencia Artificial de la Universidad de Granada.

Informa:

Que el presente trabajo, titulado ***Aplicación móvil para la detección precoz de melanomas utilizando técnicas de aprendizaje profundo***, ha sido realizado bajo su supervisión por **José Illana Lope (alumno)**, y autorizo la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expido y firmo el presente informe en Granada a 5 de septiembre de 2025 .

El director:

Carlos Gustavo Porcel Gallego (tutor)

Agradecimientos

Quiero agradecer a mis padres por haberme dado la oportunidad de estudiar lo que más me gusta, y haberme apoyado a lo largo de todo este proceso. También agradecer a todos los compañeros que he conocido a lo largo de los años, ya que todos han contribuido a que sea quien soy hoy en mayor o menor medida.

Por último quiero agradecer a los profesores que he tenido a lo largo de la carrera, por haberme brindado una formación de calidad que me permitirá trabajar de aquello que me gusta teniendo unos conocimientos sólidos.

Índice de figuras

2.1. Imagen de un melanoma	6
2.2. Comparación entre una neurona real y una artificial	7
2.3. Estructura clásica de una red neuronal convolucional	7
2.4. Operación de Max-Pooling sobre un mapa de activación	8
2.5. Arquitectura de una red U-Net	9
2.6. Estructura de un Transformer para visión	10
2.7. Principales frameworks de aprendizaje profundo	12
3.1. Planificación en cascada propuesta para el desarrollo del proyecto	16
5.1. Diagrama de flujo de datos de la aplicación	28
5.2. Imágenes obtenidas del dataset HAM10000	30
5.3. Arquitectura de la red Mobilenet	31
5.4. Diagrama de flujo dentro de la aplicación	32
5.5. Pantalla para mostrar en una ventana flotante una predicción del historial en detalle	33
5.6. Pantalla que muestra la cámara junto con tres botones, uno para tomar la fotografía, otro para abrir la galería y otro para volver al menú principal	34
5.7. Página principal de la aplicación	35
5.8. Logo de Google Colab	36
6.1. Lesiones que se encuentran en el conjunto de datos junto con sus máscaras de segmentación	41
6.2. Ejemplo de predicción tras el entrenamiento. De izquierda a derecha, se puede observar la imagen real, la máscara autentica y la máscara predicha por el modelo	43
6.3. Otro ejemplo de predicción tras el entrenamiento	44
6.4. Salida del programa tras la ultima época de entrenamiento. De izquierda a derecha: Tiempo de entrenamiento para esa época, exactitud y pérdida en el conjunto de datos de entrenamiento y en el conjunto de validación	44
6.5. Predicción del modelo clasificador	46

6.6. Otra predicción del modelo clasificador, en este caso predice maligna	46
6.7. Diagrama de la arquitectura de la aplicación	47
6.8. Pantalla principal de la aplicación	50
6.9. Pantalla para tomar imágenes de las lesiones en la piel	51
 A.1. Predicción B-CAM-1	66
A.2. Predicción B-CAM-2	67
A.3. Predicción B-CAM-3	68
A.4. Predicción B-CAM-4	69
A.5. Predicción B-CAM-5	70
A.6. Predicción B-GAL-1	71
A.7. Predicción B-GAL-2	72
A.8. Predicción B-GAL-3	73
A.9. Predicción B-GAL-4	74
A.10. Predicción B-GAL-5	75
A.11. Predicción M-GAL-1	76
A.12. Predicción M-GAL-2	77
A.13. Predicción M-GAL-3	78
A.14. Predicción M-GAL-4	79
A.15. Predicción M-GAL-5	80
A.16. Predicción M-CAM-1	81
A.17. Predicción M-CAM-2	82
A.18. Predicción M-CAM-3	83
A.19. Predicción M-CAM-4	84
A.20. Predicción M-CAM-5	85

Índice de tablas

4.1. Requisitos funcionales del sistema	20
4.2. Casos de uso propuestos para la aplicación	22
4.3. Requisitos no funcionales del sistema	23
4.4. Requisitos de datos para la aplicación	24
4.5. Capacidad de almacenamiento estimado	25
4.6. Presupuesto de desarrollo	26
7.1. Resultados de evaluación del modelo de segmentación (U-Net)	54
7.2. Resultados de evaluación del modelo de clasificación (MobileNetV3)	54
7.3. Resultados de la prueba de campo simulada con el conjunto BCN20000.	55

Índice general

1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Estructura de la memoria	2
2. Antecedentes	5
2.1. Dominio del problema	5
2.1.1. Melanoma	5
2.1.2. Redes neuronales	6
2.2. Estado del arte	11
2.2.1. Entornos de desarrollo	11
2.2.2. International Skin Imaging Collaboration	12
3. Metodología	15
4. Análisis	19
4.1. Requisitos funcionales	19
4.2. Requisitos no funcionales	22
4.3. Requisitos de datos	23
4.4. Requisitos de almacenamiento	24
4.5. Presupuesto	25
5. Diseño	27
5.1. Diseño de la arquitectura del sistema	27
5.2. Selección del conjunto de datos de entrenamiento	29
5.3. Selección de los modelos de aprendizaje profundo	30
5.4. Diseño de la interfaz de usuario y experiencia de usuario . . .	31
5.5. Diseño de la arquitectura tecnológica	35
5.5.1. Diseño de la arquitectura tecnológica para el desarrollo y entrenamiento de modelos de aprendizaje profundo	36
5.5.2. Diseño de la arquitectura tecnológica para el desarrollo frontend/ backend de la aplicación móvil	37
5.6. Diseño de la persistencia de datos locales	37

6. Implementación	39
6.1. Configuración del entorno de desarrollo para la implementación y entrenamiento de modelos de aprendizaje automático .	39
6.2. Configuración del entorno para el desarrollo de la aplicación móvil.	45
6.3. Diagrama de arquitectura	47
6.4. Desarrollo de la aplicación y lógica de negocio	48
6.4.1. Persistencia de datos con Room	48
6.4.2. Implementación de la interfaz con Jetpack Compose .	48
6.5. Implementación del pipeline de inferencia	49
7. Evaluación y fase de pruebas	53
7.1. Evaluación individual de los modelos	53
7.2. Evaluación de la aplicación en una prueba de campo simulada	54
8. Conclusiones y trabajos futuros	57
9. Bibliografía	61
A. Apéndice: Imágenes tomadas para la prueba de campo	65

Capítulo 1

Introducción

Este capítulo tiene como propósito introducir las diversas causas y motivaciones que han impulsado el desarrollo de este proyecto. Asimismo, se expone la estructura y la metodología que se seguirán para alcanzar los objetivos propuestos.

1.1. Motivación

La principal motivación para realizar este trabajo se basa en una convergencia de intereses tanto personales como sociales y tecnológicos en la detección temprana de una enfermedad que afecta a tantas personas, utilizando los avances en un campo tan a la orden del día y con tanto potencial como es la inteligencia artificial para brindar un impacto positivo a la sociedad.

El melanoma, una de las formas más agresivas que puede tomar el cáncer de piel, representa un problema que afecta a miles de personas cada año. La detección precoz de este mismo representa un factor clave en las futuras etapas del tratamiento del mismo[31]. Además, esta detección aumenta significativamente las tasas de supervivencia en los pacientes que lo sufren. Sin embargo, su diagnóstico sigue siendo una decisión subjetiva en la que influyen factores humanos como la experiencia o edad del dermatólogo. Este contexto presenta la necesidad de desarrollar herramientas que asistan tanto a los profesionales como a los pacientes en las primeras etapas de detección y categorización de este tipo de cáncer.

Por otro lado, en los últimos años hemos observado un creciente interés en el ámbito de la inteligencia artificial, con estimaciones que sitúan el valor de este mercado en 100.000 millones de dólares para 2026[21]. Este creciente interés ha llevado a la aparición de diversas tecnologías que ponen al alcance de cualquier usuario herramientas que antes estaban reservadas a un grupo reducido de personas. Esto presenta un panorama ideal para la investigación y el desarrollo por parte de cualquier usuario de herramientas que puedan

brindar un impacto positivo en distintos campos. El objetivo de este proyecto no es solo el de explorar las distintas tecnologías, tales como las redes neuronales, sino el de encontrar su aplicación y utilidad frente a problemas reales.

En último lugar, existe una motivación personal para colaborar y aportar en un ámbito en el que la tecnología puede salvar vidas. La creación de una aplicación que permita facilitar estos diagnósticos tempranos presenta tanto un reto apasionante como una oportunidad única de aplicar todo lo aprendido durante mi formación académica en un proyecto que puede mejorar la calidad de vida de las personas.

Es importante destacar que este proyecto no busca sustituir el trabajo de los dermatólogos ni demás profesionales sanitarios, sino simplemente aportar una herramienta que permita ayudar a detectar de forma temprana casos de melanoma, sirviendo así no como reemplazo, sino como apoyo para estos profesionales sanitarios, y de hecho, se nutre de su experiencia para su funcionamiento y posterior aprendizaje.

1.2. Objetivos

El objetivo principal del proyecto es el de desarrollar una aplicación móvil que permita a los usuarios tomar fotografías de su piel y, aplicando técnicas de aprendizaje profundo, ayude a clasificar estas fotografías para determinar si se trata de una lesión maligna o benigna. Este objetivo principal se desglosa en los siguientes objetivos específicos:

- Analizar el estado del arte en el ámbito de la visión por computador y el análisis de patrones.
- Buscar y analizar distintos conjuntos de datos e imágenes que puedan ser útiles en nuestro proyecto.
- Implementar distintas técnicas basadas en aprendizaje profundo que se consideren más apropiadas para la caracterización y clasificación de imágenes.
- Estudio comparativo de las técnicas implementadas con los conjuntos de datos identificados para seleccionar la más apropiada.
- Desarrollar una aplicación móvil que integre la técnica más prometedora.

1.3. Estructura de la memoria

Con el objetivo de documentar el trabajo que se ha realizado a lo largo del desarrollo de este proyecto, así como de justificar los diferentes motivos

que han llevado a tomar ciertas decisiones a lo largo del proceso de desarrollo, se propone la siguiente estructura para la memoria:

- **Capítulo 2 Antecedentes:** En esta sección se explorará el dominio del problema, además de realizar un análisis del estado del arte que nos permita saber en qué estado se encuentra y qué tecnologías nos pueden resultar útiles para nuestro desarrollo.
- **Capítulo 3 Metodología:** En este punto se abordará la metodología que se utiliza durante todo el proceso de desarrollo, así como las distintas etapas en las que lo se divide.
- **Capítulo 4 Análisis:** Aquí se realiza un análisis en profundidad de las necesidades de nuestra aplicación y se plantean una serie de requisitos (tanto funcionales como no funcionales, de almacenamiento, de datos...) así como otras necesidades que puedan desprenderse del desarrollo.
- **Capítulo 5 Diseño:** En esta sección se documentara toda la fase de diseño de la aplicación, presentando las diferentes tecnologías que se usarán así como la justificación del porqué se han seleccionado.
- **Capítulo 6 Implementación:** Aquí se explicarán los pasos que se han seguido hasta conseguir tener la aplicación funcionando.
- **Capítulo 7 Evaluación y fase de pruebas:** Aquí se evaluará la aplicación frente a distintas situaciones de uso para detectar posibles fallos en su funcionamiento o mejoras a realizar.
- **Capítulo 8 Conclusiones y trabajos futuros:** En este capítulo se presentarán las conclusiones que se han obtenido durante todo el desarrollo del proyecto, y se plantearán posibles mejoras a implementar en un futuro.

Capítulo 2

Antecedentes

En este capítulo, se realiza un análisis exhaustivo de los diferentes conceptos y términos que serán necesarios para comprender el desarrollo de este proyecto. Por un lado, se destacarán las diferentes tecnologías que están disponibles para el desarrollo de modelos de aprendizaje profundo, así como aquellas tecnologías que permiten la implementación de estos modelos en distintas aplicaciones. Este análisis proporcionará una base sólida para poder escoger las tecnologías que más beneficien al desarrollo de nuestro proyecto.

2.1. Dominio del problema

A continuación, se presenta una introducción a los principales conceptos que son de especial relevancia para el desarrollo del proyecto.

2.1.1. Melanoma

El melanoma (ver Figura 2.1) es un tipo de cáncer de piel que se produce cuando las células que dan el color bronceado a la piel (conocidas como melanocitos) empiezan a reproducirse de forma descontrolada [38]. Este tipo de cáncer de piel es especialmente peligroso, ya que tiene mayor probabilidad de extenderse a diferentes partes del cuerpo si no se realiza un diagnóstico temprano. Además, este tipo de cáncer ha aumentado su incidencia durante los últimos años, debido a diversos factores como el aumento de la exposición a la luz solar de la población o el envejecimiento de la misma [37].

Uno de los factores determinantes en las tasas de supervivencia de este tipo de cáncer es un diagnóstico temprano correcto. Este diagnóstico está ligado muchas veces a factores humanos como puede ser la experiencia del dermatólogo/a que se encarga de analizar al paciente, o los medios a los que tiene acceso el especialista, que pueden variar dependiendo de la región del mundo en la que se encuentre.



Figura 2.1: Imagen de un melanoma

La aplicación de las nuevas tecnologías ha ayudado a aumentar significativamente las tasas de éxito de diferentes procedimientos en el ámbito médico, así como a agilizar y optimizar distintos procesos. Un ejemplo de ello es la implementación de un sistema de Registros Médicos Electrónicos (EMR por sus siglas en inglés *Electronical Medical Registries*) en un hospital sueco, que permitió mejorar la atención al paciente al facilitar el acceso a la información clínica, además de reducir considerablemente el número de llamadas que recibía el personal sanitario[25].

2.1.2. Redes neuronales

En este contexto, el uso de redes neuronales para el análisis de imágenes médicas se erige como una solución que busca paliar los inconvenientes humanos anteriormente mencionados. Las redes neuronales [1] se crean con el objetivo de imitar el comportamiento que tienen las neuronas humanas (ver Figura 2.2). En el caso de las neuronas humanas, las conexiones sinápticas cambian en función de los estímulos que recibe el organismo, y de esta manera, es como se produce el aprendizaje. La redes neuronales buscan simular este comportamiento asignándole un peso a cada una de las entradas que recibe la neurona artificial, modificando el resultado de la función que calcula esa neurona. Además, estos pesos se propagan desde las neuronas de entrada a las neuronas de salida, y viceversa, lo que se conoce como *backpropagation*. La manera en la que las redes neuronales aprenden es modificando los pesos de las entradas de las neuronas. Al igual que los humanos recibimos estímulos del exterior como medio de aprendizaje, las redes neuronales también reciben estímulos, en este caso en forma de datos de entrenamiento, los cuales tienen unos valores de entrada y unos valores de salida. Teniendo ambos valores, las redes neuronales pueden ir ajustando los pesos de las conexiones entre neuronas hasta obtener el resultado deseado.

Esta tecnología abre enormes posibilidades en el campo de análisis de imágenes, ya que permite analizar miles de imágenes médicas al mismo tiem-

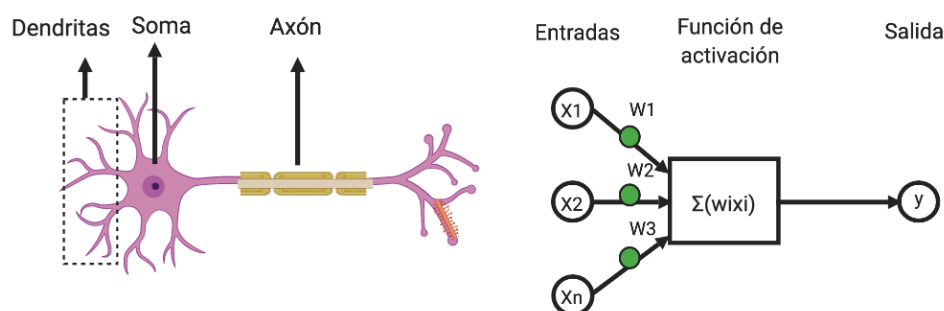


Figura 2.2: Comparación entre una neurona real y una artificial

po y detectar patrones en ellas que ayudarán a mejorar el diagnóstico de enfermedades. En campos de la medicina como la farmacología, las redes neuronales ya están acelerando los procesos de descubrimiento de fármacos permitiendo reducir el número de experimentos necesarios, los cuales serían más costosos y lentos[29].

El campo de la inteligencia artificial ha experimentado un crecimiento exponencial en los últimos años, lo que ha propiciado la aparición de multitud de herramientas que han facilitado el desarrollo de modelos de aprendizaje profundo y han permitido a cualquier usuario desarrollar los suyos propios.

En el ámbito de análisis de imágenes, las Redes Neuronales Convolucionales (CNN) han demostrado su eficacia en el reconocimiento de patrones dentro de imágenes.

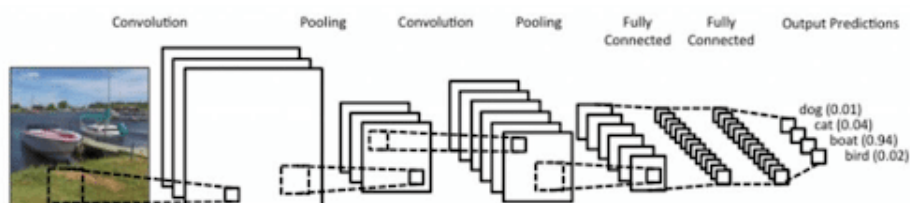


Figura 2.3: Estructura clásica de una red neuronal convolucional

Las CNNs son un tipo de red neuronal artificial que se utiliza principalmente en el campo del reconocimiento de patrones dentro de las imágenes. A diferencia de las redes neuronales tradicionales, las CNNs están diseñadas para codificar características específicas de la imagen en su arquitectura, lo que las hace más adecuadas para tareas relacionadas con imágenes y reduce los parámetros necesarios para configurar el modelo [23]. En la figura 2.3 se puede observar la estructura habitual de una CNN, la cual se divide en las siguientes capas de neuronas:

- **Capas convolucionales:** Estas capas son las capas principales de

una CNN. Utilizan pequeños filtros (también conocidos como kernels) que se desplazan por la imagen realizando operaciones de convolución (operaciones matemáticas que se realizan sobre la matriz de píxeles que representa a la imagen). Estas operaciones dan como resultado mapas de activación, que indican en qué parte de la imagen se han encontrado las características que ha detectado cada kernel.

- **Capas Pooling:** La función de estas capas es la de reducir la dimensionalidad de las capas anteriores. La operación más común en este tipo de capas es la de max-pooling, la cual se puede observar en la figura 2.4 . Mediante esta operación, se selecciona el valor máximo dentro de una región de una imagen. Con esta operación se busca reducir el número de parámetros necesarios para entrenar la red neuronal, así como hacerla más robusta frente a pequeños cambios que se puedan producir en la imagen[23].
- **Capas densas o totalmente conectadas:** Estas capas toman las salidas tanto de las capas convolucionales como de las capas de pooling y las utilizan para determinar la clasificación final.

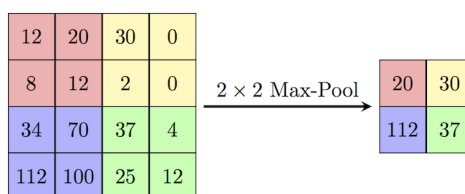


Figura 2.4: Operación de Max-Pooling sobre un mapa de activación

Esta tipo de redes neuronales son ampliamente utilizadas en el ámbito de la visión artificial debido a que han demostrado una gran eficacia para clasificar todo tipo de imágenes. Modelos como el ResNet[15] han alcanzado un rendimiento a nivel humano, con un error top-5 del 3.6 % en el ImageNet Large Scale Visual Recognition Challenge (ILSVRC)[35] de 2015, un reto que evalúa diferentes algoritmos para la detección y clasificación de objetos a gran escala. Se pueden observar diferentes casos de uso de estas redes en nuestra vida cotidiana, desde su aplicación en tecnologías de reconocimiento facial como FaceID de Apple[3], hasta el uso de ellas en sistemas de conducción autónomos, permitiendo reconocer señales, peatones y otros automóviles[2].

Otro tipo de redes neuronales que ha demostrado un gran impacto en el análisis de imágenes son las U-Net, especialmente en tareas de segmentación de imágenes. Estas redes, introducidas por Ronneberger et al.[32], toman su nombre por su arquitectura en forma de U(Figura 2.5). Este tipo

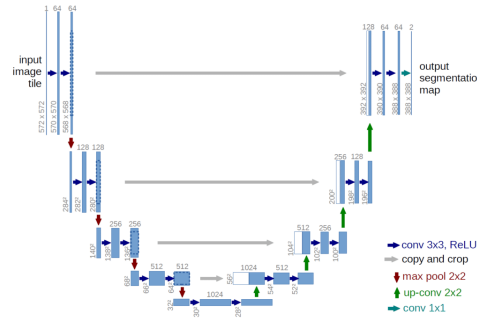


Figura 2.5: Arquitectura de una red U-Net

de arquitectura fue diseñada especialmente para la segmentación de imágenes biomédicas, por lo que destacan en tareas como la detección de tumores en imágenes de resonancia magnética, tomografías computarizadas y mamografías, permitiendo una identificación más precisa y temprana de anomalías. Su capacidad para capturar tanto los detalles locales como la estructura global de una imagen las hace especialmente efectivas en la segmentación de tejidos, vasos sanguíneos y órganos en estudios clínicos[32]. Además, su estructura en forma de U, con una fase de codificación y otra de decodificación, permite recuperar detalles espaciales que otro tipo de arquitecturas podrían perder. La arquitectura U-Net ha demostrado una eficacia sobresaliente en el ámbito del análisis de imágenes, lo que ha propiciado su adopción generalizada en diversas disciplinas médicas. Sin embargo, su uso trasciende el ámbito médico, siendo usadas también en ámbitos como el análisis de imágenes satelitales para la detección de deforestación o el seguimiento de desastres naturales.

En concreto, es de especial interés una variante de la anteriormente mencionada U-Net denominada MALUNet[34] la cual introduce mecanismos de atención múltiple y un diseño ligero que optimiza tanto la precisión como la eficiencia computacional en tareas de segmentación médica. A diferencia de la U-Net convencional, MALUNet emplea módulos de atención que permiten enfocarse en las regiones más relevantes de la imagen, mejorando la detección de características sutiles en lesiones cutáneas. El funcionamiento de MALUNet se basa en la integración de mecanismos de atención que refinan la extracción de características, permitiendo que la red enfoque su capacidad de análisis en las regiones más relevantes de la imagen. Para ello, emplea una combinación de módulos de atención espacial y de canal, que mejoran la identificación de patrones específicos en lesiones cutáneas al filtrar información irrelevante y reducir el ruido en las imágenes médicas. Además, su estructura ligera optimiza la eficiencia computacional mediante la reducción del número de parámetros y el uso de convoluciones eficientes, lo que minimiza la carga de procesamiento sin comprometer la precisión del

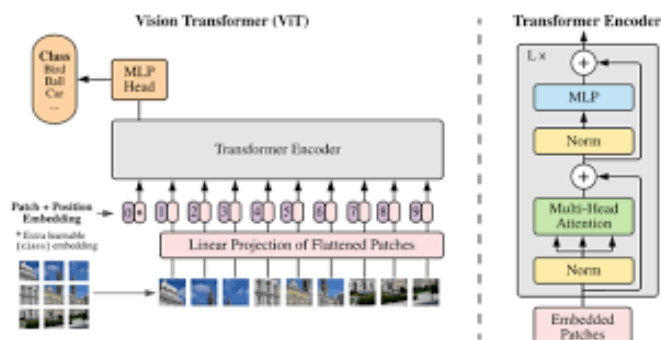


Figura 2.6: Estructura de un Transformer para visión

modelo.

En términos prácticos, MALUnet ha demostrado su efectividad en diversas aplicaciones médicas relacionadas con la segmentación de imágenes, como la detección de tumores cerebrales en resonancias magnéticas, la identificación de anomalías en radiografías pulmonares y, más específicamente, en la segmentación de lesiones cutáneas para el diagnóstico de melanomas. En estudios recientes, se ha observado que el uso de MALUnet en imágenes dermatoscópicas mejora la segmentación de los bordes de los lunares, lo que es crucial para la diferenciación entre lesiones benignas y malignas.

Además de las redes mencionadas anteriormente, existen otras opciones como las Redes Generativas Antagónicas (GANs), que han demostrado ser altamente efectivas en la mejora y síntesis de imágenes. Estas redes, compuestas por un generador y un discriminador que compiten entre sí, se utilizan en tareas como la super-resolución de imágenes médicas, la síntesis de imágenes fotorrealistas y la traducción de estilos en imágenes satelitales y artísticas[13].

Otra alternativa destacada son los Transformers para visión (ViTs), que han emergido como una opción poderosa para el análisis de imágenes sin la necesidad de convoluciones tradicionales (Figura 2.6). Estos modelos basados en mecanismos de atención han superado a las CNNs en algunas tareas de clasificación y segmentación de imágenes, especialmente en conjuntos de datos a gran escala como ImageNet[12].

Asimismo, los Autoencoders juegan un papel clave en la compresión y eliminación de ruido en imágenes, siendo utilizados en áreas como la restauración de imágenes degradadas, la detección de anomalías en imágenes médicas y la reducción de dimensionalidad en datos visuales[4].

Finalmente, arquitecturas híbridas como las ConvLSTMs[36] combinan las ventajas de las redes convolucionales con las redes recurrentes, permitiendo el análisis de secuencias de imágenes en videos y mejorando el seguimiento de objetos en aplicaciones como la conducción autónoma y la videovigilancia

inteligente.

2.2. Estado del arte

En esta sección se procederá a analizar las diferentes tecnologías que serán de utilidad para el desarrollo de la aplicación, y se procederá a realizar un análisis para comprobar las soluciones ya existentes en el ámbito de la aplicación que se planea implementar.

2.2.1. Entornos de desarrollo

Tras evaluar las diversas arquitecturas de redes neuronales potencialmente aplicables al proyecto, se concluye que su implementación implicaría una inversión de tiempo y recursos que excede los disponibles. Por ello, se plantea la necesidad de recurrir a entornos de desarrollo que optimicen los procesos e implementen el modelo de aprendizaje profundo de forma más eficiente y accesible.

Un entorno de desarrollo (framework de aquí en adelante por su nombre en inglés)[30] es una estructura conceptual, de asistencia definida que normalmente se sirve de herramientas de software tales como bibliotecas o módulos, que permiten desarrollar nuevos productos de software con una mayor facilidad.

En los últimos años han surgido diferentes frameworks en el ámbito del aprendizaje automático (ver Figura 2.7). Estos frameworks han permitido agilizar el desarrollo de modelos de aprendizaje profundo proporcionando una capa de abstracción sobre la complejidad inherente a estas arquitecturas. Gracias a esta abstracción, los desarrolladores e investigadores pueden concentrarse en la definición de la arquitectura del modelo y en la experimentación con diferentes configuraciones y datos, en lugar de tener que implementar manualmente las operaciones matemáticas de bajo nivel necesarias para el entrenamiento de redes neuronales.

Uno de los frameworks más utilizado en la actualidad es Tensorflow[39]. Desarrollado por ingenieros e investigadores del equipo de Machine Learning dentro de Google Brain, este framework aporta soluciones robustas y flexibles para la construcción y el despliegue de modelos de aprendizaje automático a gran escala. Desde su lanzamiento inicial en 2015, TensorFlow ha evolucionado constantemente, adaptándose a las últimas tendencias en investigación y a las necesidades de la comunidad. Su arquitectura está diseñada para ser eficiente y escalable, permitiendo a los usuarios entrenar modelos complejos en diversas plataformas, desde CPUs y GPUs hasta clusters distribuidos y dispositivos móviles.

Otro framework destacable es PyTorch[26]. Pytorch nace como una librería de aprendizaje automático que busca un equilibrio entre usabilidad y



Figura 2.7: Principales frameworks de aprendizaje profundo

velocidad. Su principal característica es que busca seguir un estilo de programación *Pythonic* (estilo de programación en el cual se siguen los principios semánticos del lenguaje de programación Python). Este estilo de programación facilita la depuración de código, así como su implementación. PyTorch se ha utilizado en la implementación de multitud de aplicaciones que utilizan técnicas de aprendizaje profundo, desde aplicaciones para el reconocimiento de imágenes hasta Procesadores de Lenguaje Natural (NLP por sus siglas en inglés)[8].

Cabe mencionar otros frameworks como scikit-learn[27], Keras[7]... los cuales, aunque no tan usados, también proporcionan herramientas muy poderosas para el desarrollo de modelos de aprendizaje profundo.

2.2.2. International Skin Imaging Collaboration

La International Skin Imaging Collaboration (ISIC de aquí en adelante) nace con el objetivo de reducir las muertes relacionadas con el melanoma así como reducir el número de biopsias. Para ello, busca mejorar precisión y la exactitud en la detección temprana de este tipo de cáncer estableciendo estándares en la recolección de imágenes dermatológicas, así como animando tanto a las comunidades científicas como a los propios usuarios a desarrollar proyectos de detección temprana usando aprendizaje profundo. En la actualidad, cuenta con un archivo de más de 500.000 imágenes, etiquetadas según el tipo de lesión en la piel, la edad del paciente, su sexo, etc. Además, desde 2016 se organizan retos con conjuntos de datos específicos. En un primer lugar, estos desafíos fueron ideados con la intención de mejorar la eficiencia en el diagnóstico de melanomas, así como otras lesiones en la piel. Sin embargo, conforme la comunidad ha ido creciendo, se han presentado nuevos desafíos como los de 2020, que buscaban abordar otros factores como la importancia del contexto médico en el diagnóstico.[18]

Dentro de estos conjuntos de datos que proporciona la ISIC, cabe desta-

car uno en específico. Es el conocido como HAM10000[40]. Este conjunto de datos está compuesto por más de 10.000 imágenes recopiladas a lo largo de 20 años de dos fuentes, el Departamento de Dermatología de la Universidad Médica de Viena (Austria) y la práctica privada del doctor Cliff Rosendal en Queensland (Australia). Este conjunto de datos ha sido utilizado en diferentes retos que ha propuesto la ISIC, como el de 2018[9], en el cual se pedía a los participantes que, analizando tanto la lesión como sus características (pigmentación, forma, etc), determinasen el tipo de lesión de las que se trataba. Este conjunto de datos es especialmente interesante, ya que las imágenes que se encuentran en el han sido obtenidas de diferentes poblaciones, lo que permitirá a los modelos de aprendizaje automático generalizar mejor ya que tendrán más ejemplos de lesiones en diferentes tipos de pieles.

Otro conjunto de datos que puede resultar interesante es el BCN20000. Este conjunto de datos contiene imágenes recopiladas entre los años 2010 y 2016 en el Hospital Clínic de Barcelona. Este conjunto es interesante también ya que, aunque la población de la que se han obtenido las imágenes no es tan variada como en el HAM10000, contiene un mayor número de imágenes además enfocarse en documentar lesiones difíciles de diagnosticar como podrían ser lesiones hipopigmentadas (lesiones cuyo pigmento no es visible o es menos visible de lo que debería) así como lesiones en zonas complicadas de diagnosticar, como podrían ser uñas o mucosas[10]. Estas características podrían ayudar a los modelos de aprendizaje automático a encontrar características dentro de las imágenes que usando otros conjuntos no serían capaces de detectar.

Como se ha mencionado anteriormente, la ISIC ha organizado diferentes competiciones a lo largo de su historia que han permitido la aparición de tecnologías que pueden resultar de especial interés para el desarrollo de nuestro proyecto, como la ya mencionada MALUnet. Sin embargo, no se ha aportado ninguna solución que permita trasladar estos resultados a dispositivos móviles, acercándolo así al un mayor número de usuarios.

Capítulo 3

Metodología

En este capítulo se presenta la metodología que guiará el desarrollo del proyecto, definiendo el enfoque global y las estrategias que permitirán abordar la complejidad inherente al análisis de lesiones cutáneas para determinar su carácter maligno o benigno. Se expone un plan estructurado que abarca desde la recolección y preprocesamiento de datos hasta el desarrollo, validación y evaluación del modelo de análisis, estableciendo además las bases para la integración del mismo en la aplicación final. Este enfoque metodológico no solo garantiza un proceso riguroso y replicable, sino que también sienta las bases para los capítulos posteriores, en los cuales se profundizará en el análisis, diseño e implementación de la solución. Con ello, se busca asegurar que cada fase del proyecto contribuya de manera coherente y eficiente a la consecución de los objetivos planteados, haciendo especial hincapié en la calidad de los datos, la robustez del modelo y la gestión ética y segura de la información médica.

El primer paso para asegurar un proceso de desarrollo correcto y estructurado es definir el tipo de metodología que se usará a lo largo de la realización del proyecto. Esto permitirá dividir el desarrollo en fases cuyos objetivos están bien definidos, y permitirá optimizar la gestión del tiempo y los recursos, asegurando un flujo de trabajo bien estructurado y predecible. Para este proyecto, se ha seleccionado el modelo en cascada, dado que permite una planificación detallada desde el inicio y una ejecución secuencial de las distintas fases del desarrollo.

Este enfoque se basa en la división del proceso en etapas claramente definidas, como el análisis de requisitos, el diseño, la implementación, las pruebas, la integración y el mantenimiento. Cada fase debe completarse antes de pasar a la siguiente, lo que garantiza un control riguroso del progreso y minimiza la incertidumbre durante el desarrollo. Gracias a esta estructura, se puede establecer una documentación exhaustiva desde las primeras etapas, facilitando la trazabilidad y asegurando que todos los requisitos del proyecto sean cubiertos de manera sistemática[45]. Además, se ha seleccio-

nado este enfoque puesto que el desarrollo se realizará por una única persona y su idea claramente definida dificultaría la división en iteraciones, como las usadas en metodologías SCRUM.

Además, el modelo en cascada es especialmente adecuado para proyectos con requisitos bien definidos desde el inicio y donde se busca minimizar la necesidad de cambios durante el desarrollo. Esta metodología permitirá asegurar una implementación sólida y reducir riesgos, al garantizar que cada fase sea validada antes de avanzar a la siguiente.

En primer lugar, debemos definir las diferentes fases que tendrá nuestro desarrollo, establecer unos objetivos bien definidos para la consecución de cada una de estas, y determinar el orden en el que se llevarán a cabo, siguiendo un enfoque de desarrollo en cascada.

A grandes rasgos, el proceso se dividirá en diferentes etapas que incluirán el análisis de requisitos, el diseño e implementación de la aplicación así como la realización de pruebas de la misma. Cada una de estas fases tiene una gran importancia en el proceso de desarrollo y asegurará la calidad del producto final.

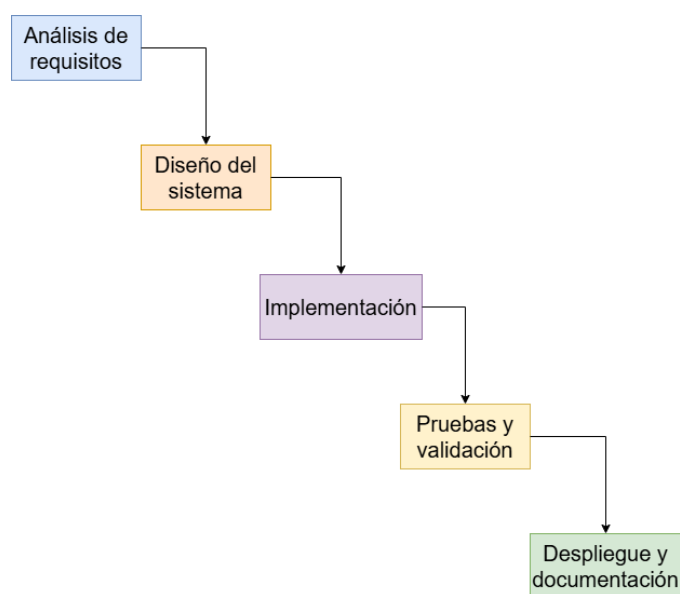


Figura 3.1: Planificación en cascada propuesta para el desarrollo del proyecto

En la figura 3.1 se puede observar un diagrama que muestra cada una de las fases de desarrollo. A continuación, se explicará de una forma resumida el objetivo de cada una de las fases:

- **Análisis de requisitos:** En esta primera fase, se plantearán los objetivos del proyecto, los diferentes requisitos (funcionales, no funcionales, de almacenamiento...) necesarios para nuestra aplicación y la búsqueda

da de conjuntos de datos y herramientas que puedan ser de utilidad en nuestro desarrollo.

- **Diseño del sistema:** En esta fase buscamos definir la estructura del sistema y desarrollar el modelo de aprendizaje profundo que permitirá a nuestra aplicación analizar las imágenes. Además, diseñaremos un primer prototipo de la interfaz, estableciendo los elementos clave para su funcionamiento y usabilidad.
- **Implementación:** A lo largo de esta fase entrenaremos el modelo diseñado en la anterior e implementaremos la aplicación que habíamos diseñado previamente.
- **Pruebas y validación:** Esta fase estará dedicada a probar la implementación realizada en la fase anterior y asegurar el correcto funcionamiento de las distintas partes de la aplicación.
- **Despliegue y documentación:** En esta última fase, se desplegará en un entorno accesible para su uso y se documentarán los futuros cambios o mejoras a realizar en el proyecto.

Este esquema metodológico nos permitirá avanzar de manera estructurada y organizada, asegurando que cada fase se complete antes de pasar a la siguiente. Al seguir este enfoque, buscamos garantizar la calidad del sistema final, optimizando tanto el rendimiento del modelo de aprendizaje profundo como la usabilidad de la aplicación. En los capítulos posteriores, se detallará cada una de estas fases en profundidad, abordando los aspectos técnicos y las decisiones tomadas en cada etapa del desarrollo.

Capítulo 4

Análisis

En este capítulo, se realizará un análisis detallado de los diferentes requisitos y elementos necesarios para el desarrollo de nuestro proyecto. En primer lugar, se definirán los requisitos funcionales y no funcionales que definirán el comportamiento de nuestra aplicación.

Posteriormente, se analizarán las especificaciones de datos que nuestro proyecto necesitará almacenar, recopilar, analizar y presentar. Esta fase es crucial, ya que el funcionamiento de los modelos de aprendizaje profundo depende en gran medida de la calidad de los datos de entrenamiento. También se estudiarán las herramientas que nos sean de mayor utilidad en nuestro desarrollo.

En conclusión, este capítulo sienta las bases para las siguientes fases del desarrollo y nos proporciona una estructura que nos permitirá realizar una implementación eficiente del sistema.

4.1. Requisitos funcionales

Los requisitos funcionales describen detalladamente las funciones que un sistema debe realizar para cumplir con las necesidades del usuario[46]. Definir correctamente estos requisitos es esencial para ofrecer una aplicación que cumpla con las expectativas del usuario y garantice su correcto funcionamiento. Además, estos requisitos guiarán el diseño e implementación de la aplicación, proporcionando una base para estructurar la arquitectura del software y elegir las tecnologías más adecuadas para nuestra aplicación.

En la tabla 4.1 se muestran todos los requisitos funcionales planteados para nuestra aplicación, junto con un comentario describiendo en profundidad cada uno de ellos.

ID requisito	Nombre	Descripción del requisito
RF-001	Captura de imagen	La aplicación permitirá a los usuarios capturar imágenes de sus lunares.
RF-002	Subida de la imagen	La aplicación permitirá a los usuarios subir imágenes desde la galería de su dispositivo.
RF-003	Preprocesamiento de la imagen	La aplicación debe procesar la imagen capturada/seleccionada para mejorar la calidad del análisis. Las imágenes serán procesadas desde el propio dispositivo para mantener la privacidad del usuario.
RF-004	Análisis de la imagen	El sistema analizará la imagen proporcionada por el usuario mediante un algoritmo de aprendizaje automático para detectar características en ella que le permitan predecir si se trata de un lunar benigno o maligno.
RF-005	Emisión de resultados	Una vez determinado el tipo de lesión, el sistema informará al usuario de su predicción.
RF-006	Información de interés	El sistema debe proporcionar al usuario información de interés sobre los melanomas.
RF-007	Almacenamiento de imágenes	Las imágenes se almacenarán en el dispositivo del usuario, sin uso de ningún sistema de almacenamiento externo.
RF-008	Historial de predicciones	El usuario podrá visualizar las predicciones pasadas que ha realizado, así como eliminarlas para liberar espacio.

Tabla 4.1: Requisitos funcionales del sistema

Además de los requisitos funcionales, es esencial definir los casos de uso. Un caso de uso es la descripción de una acción o actividad que realiza el usuario[44]. Definirlos correctamente es esencial, ya que nos permitirá establecer qué acciones estarán disponibles y cuál deberá ser el flujo de datos en

ella.

En la tabla 4.2 se muestran los diferentes casos de uso que se han propuesto para la aplicación.

ID	Caso de uso	Actores principales	Descripción	Requisitos funcionales relacionados
CU-001	Captura de imagen de un lunar	Usuario	El usuario captura una imagen de un lunar en su piel usando la cámara del dispositivo	RF-001
CU-002	Selección imagen lunar	Usuario	El usuario sube una imagen de la galería del dispositivo móvil	RF-002
CU-003	Análisis de la imagen	Usuario, sistema	El usuario inicia el análisis de la imagen del lunar capturada o seleccionada. La aplicación procesa la imagen y utiliza un algoritmo de aprendizaje automático para evaluar el riesgo de malignidad.	RF-003, RF-004
CU-004	Visualización de resultados	Usuario	El usuario visualiza la predicción que ha realizado el algoritmo, junto con un porcentaje de seguridad en esa predicción. Si la lesión es clasificada como maligna y el porcentaje de seguridad es elevado, el usuario verá un aviso para consultar esa lesión con su médico.	RF-005
CU-005	Acceder a información educativa	Usuario	El usuario accederá a información de interés sobre los melanomas tras pulsar un botón en la interfaz	RF-006

Continúa en la siguiente página

ID	Caso de uso	Actores principales	Descripción	Requisitos funcionales relacionados
CU-006	Acceso al historial de predicciones	Usuario	El usuario accederá al historial de predicciones que ha realizado con anterioridad, para revisar detalles como la imagen original, la predicción que realizó el sistema o el porcentaje de seguridad.	RF-008

Tabla 4.2: Casos de uso propuestos para la aplicación

4.2. Requisitos no funcionales

Los requisitos no funcionales definen las características que nuestro sistema debe cumplir para garantizar su seguridad, estabilidad, eficiencia y escalabilidad[47]. Los requisitos funcionales no definen qué debe hacer el sistema, sino la manera de hacerlo que asegure una experiencia de calidad al usuario. Son esenciales para garantizar que la aplicación funcione de una manera estable, además de hacerlo de forma segura.

En la tabla 4.3 se detallan los requisitos no funcionales que deberá tener nuestra aplicación.

ID requisito	Nombre	Descripción del requisito
RNF-001	Rendimiento	La aplicación debe analizar la imagen y proporcionar un resultado en un tiempo máximo de 5 segundos.
RNF-002	Usabilidad	La aplicación debe ser intuitiva para todo tipo de usuarios, independientemente de su nivel de manejo tecnológico.

Continúa en la siguiente página

ID Requisito	Nombre	Descripción del Requisito
RNF-003	Seguridad	La aplicación debe proteger la privacidad de los datos del usuario, incluyendo las imágenes de los lunares y los resultados de los análisis. Se deben implementar medidas de seguridad para evitar el acceso no autorizado, la pérdida de datos y la divulgación.
RNF-004	Análisis de la imagen	El sistema analizará la imagen proporcionada por el usuario mediante un algoritmo de aprendizaje automático para detectar características en ella que le permitan predecir si se trata de un lunar benigno o maligno.
RNF-005	Mantenibilidad	La aplicación debe ser fácil de mantener y actualizar. El código debe ser modular, estar bien documentado y seguir las mejores prácticas de desarrollo de software.
RNF-006	Eficiencia energética	La aplicación debe consumir la menor batería posible en el dispositivo móvil.

Tabla 4.3: Requisitos no funcionales del sistema

Gracias a estos requisitos, aseguraremos que nuestra aplicación ofrezca una experiencia segura y confiable para los usuarios, garantizando que todas las funcionalidades clave operen de manera eficiente y sin errores. Además, una correcta definición de estos requisitos nos permitirá desarrollar un sistema intuitivo, accesible y adaptado a las necesidades del usuario, asegurando un flujo de uso claro y una interacción fluida con la aplicación.

4.3. Requisitos de datos

Los requisitos de datos establecen qué tipo de información manejará la aplicación, en qué formato se almacenará y cómo se procesará para garantizar su correcto funcionamiento. Estos requisitos son fundamentales porque determinan cómo se capturan, almacenan y presentan los datos dentro del sistema. Definirlos con precisión es clave para asegurarnos de que la aplicación pueda gestionar la información de forma eficiente, ya sea al recibir

imágenes para su análisis, procesarlas con el modelo de inteligencia artificial o mostrar los resultados al usuario[43].

En este proyecto, la definición precisa de requisitos de datos resulta crucial, ya que no se limitará al procesamiento de texto, sino que también se incluirán archivos multimedia, como imágenes. A continuación, en la tabla 4.4, se establecen los requisitos de datos que estableceremos para nuestra aplicación:

ID requisito	Nombre	Descripción del requisito
RD-001	Formato de imagen	La aplicación debe aceptar imágenes en formatos estándar como JPEG, PNG o HEIC, asegurando la compatibilidad con la mayoría de los dispositivos.
RD-002	Resolución mínima	Las imágenes deben tener una resolución mínima de 256x256 píxeles para garantizar que el análisis realizado por el modelo sea preciso y fiable.
RD-003	Verificación de formato de la imagen	La aplicación implementará mecanismos para asegurar que todas las imágenes tienen un formato estandarizado (forma, resolución, etc) para asegurar que puedan ser analizadas de forma correcta por el modelo.

Tabla 4.4: Requisitos de datos para la aplicación

Con estos requisitos aseguraremos que las imágenes enviadas por los usuarios cumplen unos criterios mínimos tanto en formato como en calidad para ser analizadas por el modelo de aprendizaje automático. Además, también se garantizará que el sistema envíe las predicciones en un formato estandarizado.

4.4. Requisitos de almacenamiento

En esta sección definiremos los requisitos de almacenamiento. Definir estos requisitos nos permitirá establecer un marco sólido para gestionar, proteger y mantener de manera eficiente toda la información que genera y utiliza la aplicación. Al detallar cómo, dónde y en qué formato se guardarán los datos, aseguraremos no solo el cumplimiento de normativas legales y estándares de la industria, sino también la integridad y disponibilidad de la información. La adecuada planificación del almacenamiento es crucial para

optimizar el rendimiento del sistema, facilitando la recuperación rápida de datos y permitiendo futuras ampliaciones o migraciones sin contratiempos.

Teniendo en cuenta que todo el procesamiento y el almacenamiento se realizará dentro del propio dispositivo móvil, es crucial tener unos requisitos de almacenamiento bien definidos, puesto que los dispositivos móviles no cuentan con la capacidad de procesamiento ni de almacenamiento del que pueden disponer otros dispositivos de uso diario como puede ser un ordenador portátil o un ordenador de sobremesa.

En la tabla 4.4 se especifican las diferentes estimaciones de tamaño que se han realizado para diferentes elementos que poseerá la aplicación. Estas estimaciones otorgan una aproximación de la capacidad de almacenamiento que requerirá la aplicación.

Componente	Formato / Tipo	Tamaño estimado
Imágenes	JPEG de 256x256 píxeles.	75 KB por imagen (el número total es variable y gestionado por el usuario).
Modelos de aprendizaje profundo	Modelos .tflite optimizados (U-Net para segmentación y MobileNet para clasificación).	U-Net: 10 MB MobileNet: 4 MB
Almacenamiento de datos (predicciones)	Base de datos relacional SQLite. Cada registro contiene imagen original, máscara y predicción.	350 KB por registro.

Tabla 4.5: Capacidad de almacenamiento estimado

4.5. Presupuesto

En esta sección se detalla el presupuesto estimado para el desarrollo completo del proyecto, incluyendo tanto los dispositivos que se han utilizado a lo largo del desarrollo del proyecto como las herramientas por las cuales habría que desembolsar algún tipo de cantidad. En la tabla 4.6 se puede observar el presupuesto estimado para este proyecto, teniendo en cuenta el material utilizado, las horas invertidas en el desarrollo, etc.

Herramienta	Detalles	Coste aproximado
Equipo de desarrollo software	Ordenador personal	0€ (previamente adquirido)
Equipo de desarrollo móvil	Telefono Android personal	0€ (previamente adquirido)
Plataforma de desarrollo en la nube para modelos de aprendizaje automático	Google Collab: 10€ por 100 unidades de computación (la máquina con mayor capacidad de computación consume unas 8.5 unidades de computación a la hora)	10€-20€
Entornos de desarrollo móvil	Android Studio	0€ (disponible de forma gratuita)
Publicación en las tiendas de aplicaciones móviles	Google Play Store	25€ (coste único de registro como desarrollador)
Trabajo realizado (300 horas)	Trabajo personal realizado, estimando el coste en unos 15€/hora trabajada para un ingeniero informático junior	4.500€
Coste total del proyecto estimado	Suma total de todos los costes estimados	4.550€

Tabla 4.6: Presupuesto de desarrollo

Capítulo 5

Diseño

Una vez establecidos todos los requisitos de la aplicación, en este capítulo se describe a la fase de diseño. A lo largo de esta fase, se diseña toda la arquitectura necesaria para que el sistema cumpla su objetivo. A lo largo de este capítulo se desglosan las distintas fases de diseño involucradas en el desarrollo del proyecto, así como las diferentes decisiones de diseño que se han tomado, con su correspondiente explicación y justificación.

5.1. Diseño de la arquitectura del sistema

En primer lugar, se debe diseñar la arquitectura que tendrá el sistema. Esto nos permitirá controlar el funcionamiento interno que se producirá dentro de la aplicación, pudiendo seleccionar los mejores modelos de aprendizaje profundo para llevar a cabo la tarea.

En esta fase se debe establecer el flujo completo de la aplicación, desde el momento en el que el usuario toma una foto de la lesión hasta que recibe una predicción del estado de su lesión. Se debe tener en cuenta tanto el tratamiento que se hace de las imágenes recibidas por parte del usuario como las entradas y salidas que recibirá el modelo o modelos de aprendizaje profundo.

El flujo establecido para la aplicación se detalla en el diagrama mostrado en la figura 5.1.

A continuación, se explican los pasos tomados y su justificación.

- **Subida de imágenes desde la galería o desde la propia cámara del dispositivo:** El primer paso del usuario será el de subir las imágenes, ya sea desde la galería del dispositivo o desde la propia cámara. En el caso en el que el usuario decida subir las imágenes desde la propia cámara del dispositivo, se ha añadido un paso extra que consistirá en recortar la imagen con las dimensiones de una región cuadrada que el usuario podrá observar antes de tomar la foto con el objetivo de ayudarle a encuadrar mejor la lesión en la piel. En el caso de que el

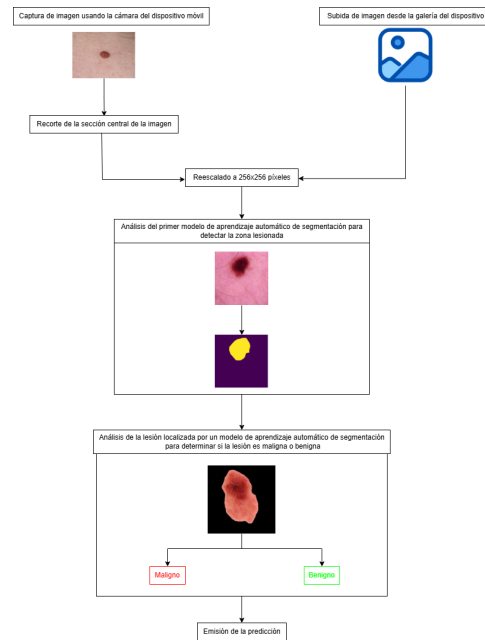


Figura 5.1: Diagrama de flujo de datos de la aplicación

usuario decida tomar una foto de la galería del dispositivo este paso no se tendrá en cuenta ya que el usuario tiene la capacidad de editarla antes de subirla utilizando herramientas de edición de imagen.

- **Reescalado a 256x256 píxeles:** Debido a que este tamaño ha sido el elegido para entrenar a los modelos, se deben reescalar todas las fotografías a este formato antes de ser enviadas a los modelos de aprendizaje automático.
- **Análisis del primer modelo de aprendizaje automático de segmentación para detectar la zona lesionada:** La imagen reescalada será analizada por un primer modelo que devolverá una máscara delimitando la lesión en la fotografía. Aunque se puede considerar que sería más óptimo mantener toda la información de la imagen, al tratarse de una aplicación que no va a recibir una entrada estandarizada debido a la variedad de cámaras de las que disponen los distintos tipos de dispositivos móviles que existen actualmente, se ha optado por un enfoque en el que la imagen analizada será únicamente la zona de la lesión en la piel.
- **Análisis de la lesión localizada por un modelo de aprendizaje automático de segmentación para determinar si la lesión es maligna o benigna:** Esta imagen con la zona de la lesión delimitada será pasada a un segundo modelo de inteligencia artificial que será el

encargado de determinar si se trata de una lesión maligna o benigna. El modelo emitirá unos porcentajes de seguridad para cada categoría, por ejemplo benigna 40 % maligna 60 %, dependiendo del tipo de lesión que determine que es más probable que contenga la imagen.

- **Emisión de resultados:** El usuario verá por pantalla la imagen original, la imagen con la lesión delimitada y la predicción, así como su porcentaje de confianza.

Gracias a este flujo de información a lo largo del sistema, los modelos de aprendizaje automático recibirán un flujo de imágenes estandarizado de tal manera que siempre analicen imágenes con el mismo formato, permitiendo un funcionamiento correcto de los algoritmos de aprendizaje profundo.

5.2. Selección del conjunto de datos de entrenamiento

Una parte crucial en el diseño de aplicaciones que utilizan modelos de aprendizaje profundo es la selección de un buen conjunto de datos para el entrenamiento de estos modelos. Este conjunto de datos será del cual se nutran los modelos de aprendizaje automático para extraer las características que les permitirán emitir una predicción.

La selección de un conjunto de datos libre de sesgos es un requisito indispensable. Estos sesgos pueden alterar significativamente los resultados y conducir a modelos con comportamientos injustos o discriminatorios. Esto implica un análisis riguroso de la distribución de las categorías para garantizar que estén balanceadas. De igual modo, es crucial identificar variables latentes —como el género o la procedencia— que puedan influir negativamente en la capacidad de generalización del modelo. Por tanto, un tratamiento cuidadoso de los datos en esta fase es esencial para construir aplicaciones robustas, efectivas y equitativas.

Tras analizar los distintos tipos de conjuntos de datos disponibles, se ha seleccionado el conjunto de datos HAM10000[40] (Figura 5.2) como conjunto de entrenamiento. El conjunto contiene un número significativo de muestras de todas las lesiones relevantes dentro del espectro de lesiones cutáneas.

Sin embargo, debido al diseño del sistema, para el entrenamiento del primer modelo necesitaremos un conjunto de datos que contenga las máscaras de segmentación de las diferentes lesiones.

Debido a que el conjunto original no contiene los datos necesarios para el entrenamiento, usaremos uno que se encuentra en la plataforma Kaggle. Kaggle es un entorno en línea especializado en ciencias de datos y aprendizaje automático, lanzado en 2010, que ofrece una amplia variedad de conjuntos de datos, competición de modelos y herramientas colaborativas para investigadores y desarrolladores[19]. Además de proporcionar una gran cantidad

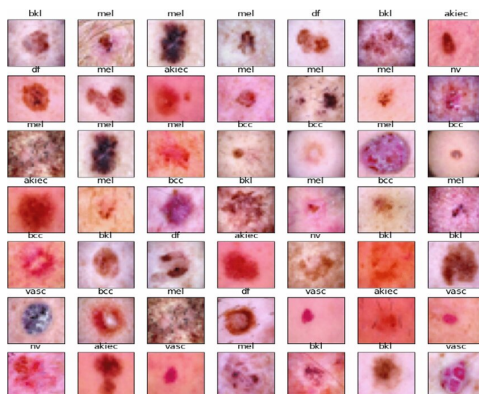


Figura 5.2: Imágenes obtenidas del dataset HAM10000

de conjuntos de datos, dispone de una librería en Python que permite una manera sencilla de descargar y almacenar los conjuntos de datos que se almacenan en la plataforma.

Para el desarrollo de la aplicación, hemos seleccionado un conjunto de datos que contiene los avances realizados por Philipp Tschandl et al en su artículo "Human-computer collaboration for skin cancer recognition"[41]. Este artículo contiene un conjunto de datos con las máscaras de segmentación de la mayor parte de las lesiones contenidas en el conjunto de datos original de HAM10000.

5.3. Selección de los modelos de aprendizaje profundo

La elección de los modelos de aprendizaje profundo es crucial, ya que serán el núcleo de la aplicación, puesto que se encargarán de analizar y clasificar las diferentes imágenes proporcionadas por el usuario. Para ello, se ha llevado a cabo un análisis exhaustivo sobre los diferentes estudios académicos existentes sobre el tema.

Según la estructura planteada en la sección anterior(ver figura 5.1), se necesitan dos modelos. Uno encargado de la segmentación de las lesiones y otro encargado de la clasificación de esas imágenes previamente segmentadas.

La arquitectura del modelo escogido para la segmentación de las imágenes es la arquitectura U-Net[32]. Aunque hubiese otras opciones más enfocadas a la tarea como podría ser la arquitectura MALUnet[34], la arquitectura U-Net presenta mayor robustez frente a la diferencia de características que pueden presentar las distintas cámaras de los dispositivos móviles frente a otras arquitecturas que están más pensadas para analizar imágenes tomadas con dispositivos profesionales de análisis dermatológico y por lo tanto

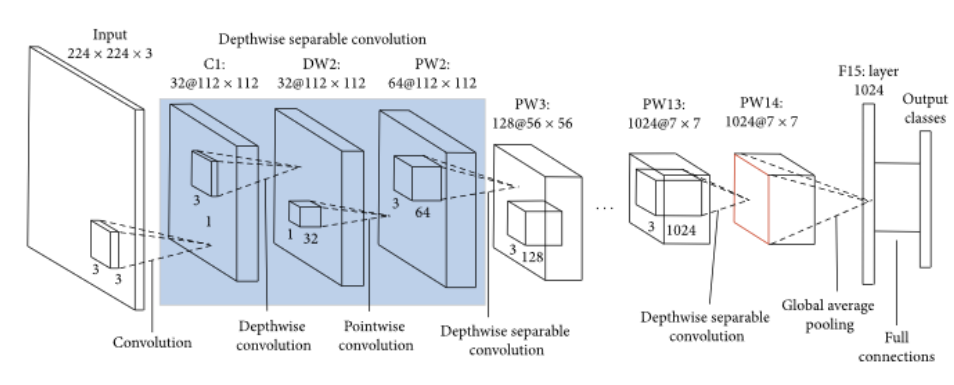


Figura 5.3: Arquitectura de la red Mobilenet

pueden presentar un peor desempeño a la hora de analizar imágenes de peor calidad como pueden ser las tomadas con dispositivos móviles. Además, la arquitectura U-Net se trata de una arquitectura ligera, es decir, la ejecución de este modelo no requiere grandes capacidades de computación, ideal para la ejecución en dispositivos móviles, los cuales poseen una capacidad de computación menor que otro tipo de dispositivos como podrían ser ordenadores de sobremesa, servidores en la nube, etc.

En cuanto al segundo modelo, hemos de tener en cuenta las mismas consideraciones tenidas en cuenta en el momento de elección del primer modelo.

Tras analizar diferentes publicaciones, se ha optado por utilizar el modelo MobilenetV3[16]. MobilenetV3 es el último modelo (a fecha de 11 de junio de 2025) de la familia de las arquitecturas Mobilenet[17]. La arquitectura Mobilenet está especialmente diseñada para dispositivos móviles (Figura 5.3), porque priorizan la eficiencia, el tamaño reducido del modelo y la baja latencia en plataformas con recursos computacionales limitados. Respecto a MobileNetV3, el cual usa el algoritmo NetAdapt para optimizar su ejecución, combina técnicas de búsqueda complementarias con avances arquitectónicos novedosos para lograr alta precisión y baja latencia, mejorando la experiencia del usuario y prolongando la duración de la batería al reducir el consumo de energía en aplicaciones basadas en redes neuronales.

5.4. Diseño de la interfaz de usuario y experiencia de usuario

En esta sección se detalla el proceso de diseño de la interfaz de usuario de la aplicación. El objetivo de esta sección no es solo asegurar un correcto funcionamiento de la aplicación, sino también hacerla intuitiva, de forma que el usuario identifique en cada momento qué acción debe tomar.

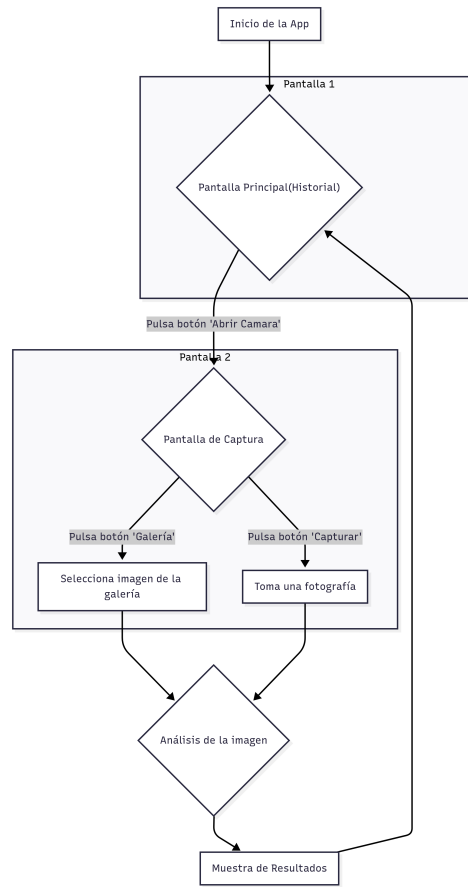


Figura 5.4: Diagrama de flujo dentro de la aplicación

Uno de los objetivos principales de la aplicación es la simplicidad, por lo tanto, se ha optado por simplificar al máximo los flujos dentro de la aplicación, de forma que las predicciones se puedan hacer de una forma intuitiva y cómoda para el usuario.

Para ello, se ha elaborado un diagrama (Figura 5.4) en el cual se pueden observar los distintos flujos que se llevarán a cabo dentro de la aplicación.

Además del diagrama de flujo, se han diseñado también una serie de esquemas de interfaz para tener un primer esbozo de las pantallas que formarán parte de la aplicación. Estos esquemas nos permitirán situar los distintos elementos (botones, listas, etc) que formarán parte de las pantallas de la aplicación, así como obtener un mapa visual de la estructura completa de la aplicación, lo que facilita la validación de la experiencia de usuario y sirve como guía fundamental durante la fase de implementación.

La página principal de la aplicación (ver figura 5.7) contendrá un botón para abrir la cámara y poder tomar fotos de las lesiones, un historial de

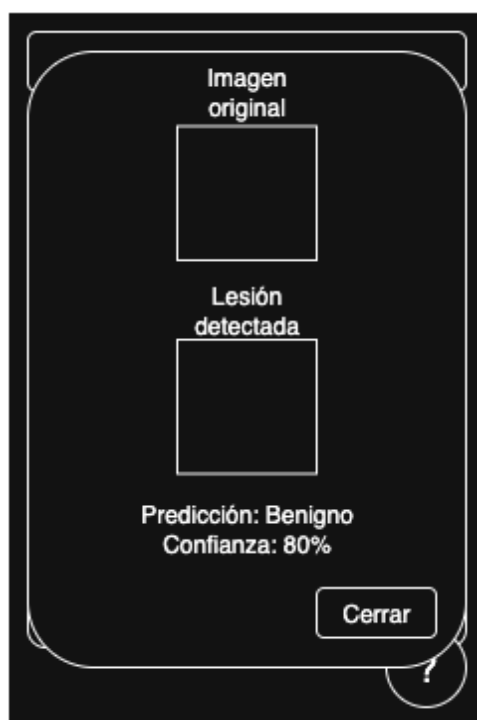


Figura 5.5: Pantalla para mostrar en una ventana flotante una predicción del historial en detalle

las predicciones hechas con anterioridad (con un botón para eliminar las predicciones que no se deseen conservar) en las cuales se pueden pulsar para consultar más información sobre la predicción mostrando una ventana flotante como se puede ver en la figura 5.5, así como un botón flotante con forma de interrogación que nos permitirá acceder a información de interés sobre los melanomas así y otro tipo de lesiones en la piel.

Una vez se accede a la cámara (Figura 5.6), se visualiza una vista que nos muestra la cámara con un cuadro que ayudará al usuario a tomar una foto centrada de la lesión. Además, habrá un botón para volver al menú principal y otro para acceder a la galería y subir fotos que se encuentren en la memoria del dispositivo móvil. Un tercer botón permitirá al usuario tomar una foto de la lesión. Una vez se pulse ese botón, el usuario verá un icono de carga, mientras la aplicación analiza la imagen tomada. Una vez se termine de analizar, se mostrará una vista previa de la predicción junto con la zona en la que se ha detectado la lesión recortada, que permitirá al usuario decidir si quiere guardar la predicción en el historial o prefiere descartarla.

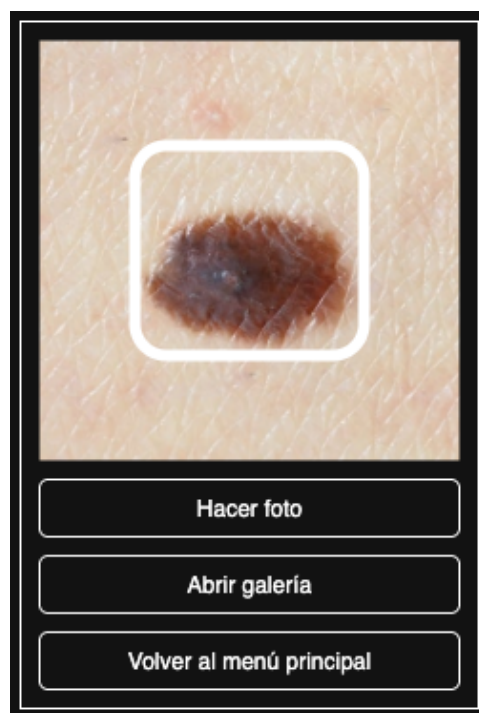


Figura 5.6: Pantalla que muestra la cámara junto con tres botones, uno para tomar la fotografía, otro para abrir la galería y otro para volver al menú principal



Figura 5.7: Página principal de la aplicación

5.5. Diseño de la arquitectura tecnológica

Teniendo en cuenta los diseños planteados anteriormente, en esta sección se analizan las tecnologías a utilizar para el desarrollo de la aplicación. Este planteamiento se dividirá principalmente en dos secciones: Arquitectura tecnológica para el desarrollo y entrenamiento de modelos de aprendizaje profundo y arquitectura tecnológica para el desarrollo frontend/backend de la aplicación móvil.

Esta distinción se debe principalmente a que el entrenamiento de los modelos de aprendizaje profundo debe realizarse en máquinas con una capacidad de computación superior a la que ofrecen los dispositivos móviles. Mientras que el entrenamiento es un proceso computacionalmente muy costoso que requiere hardware especializado (como Unidades de Procesamiento Gráfico-GPUs) y el manejo de grandes volúmenes de datos, la ejecución del modelo en la aplicación (conocida como inferencia) debe ser ligera, rápida y eficiente para garantizar una buena experiencia de usuario y un consumo de batería razonable.



Figura 5.8: Logo de Google Colab

5.5.1. Diseño de la arquitectura tecnológica para el desarrollo y entrenamiento de modelos de aprendizaje profundo

En cuanto a las tecnologías usadas para desarrollar los modelos de aprendizaje profundo, se ha seleccionado como herramienta de desarrollo Google Colab (Figura 5.8). Esta plataforma, que se originó como un proyecto interno de Google y se hizo pública en 2017, es un servicio en la nube basado en los cuadernos de Jupyter que permite escribir y ejecutar código Python a través de un navegador [5].

La elección de esta herramienta se fundamenta en varias ventajas clave que son especialmente relevantes para el desarrollo de modelos de aprendizaje profundo. La más significativa es el acceso gratuito a recursos de hardware de alto rendimiento, como GPU, lo cual es indispensable para acelerar el entrenamiento de redes neuronales complejas y reducir los tiempos de experimentación de días a horas [5]. Además, Colab ofrece un entorno preconfigurado con las principales librerías de ciencia de datos, como son TensorFlow[39], Scikit learn[27] entre otras, lo que hace el desarrollo de modelos de aprendizaje automático mucho más sencillo.

Esta herramienta aprovecha la estructura de los Jupyter Notebook[28]. Los Jupyter Notebook permiten combinar celdas de código que se pueden ejecutar de forma independiente con celdas de texto enriquecido, lo que permite documentar el código al mismo tiempo que ejecutar fragmentos del programa de forma independiente, permitiendo modificar distintos parámetros en ciertas partes del código y comprobar como afectan al programa sin tener que ejecutarlo desde cero.

El lenguaje elegido para la parte de diseño y entrenamiento de modelos de aprendizaje profundo será por tanto Python[42]. El lenguaje elegido para la parte de diseño y entrenamiento de modelos de aprendizaje profundo será por tanto Python [42]. La elección de Python se debe a que es, actualmente, el lenguaje más extendido para el desarrollo en inteligencia artificial.

Dentro del ecosistema de Python, se ha optado por TensorFlow [39] como framework para construir los modelos. Al ser una de las herramientas más conocidas y utilizadas del mundo, desarrollada por Google, cuenta con un

gran soporte y una comunidad muy activa. La razón principal de su elección es que integra Keras como su interfaz principal. Keras permite construir y probar modelos de redes neuronales de una forma mucho más simple y directa, lo que nos ha permitido experimentar con diferentes arquitecturas de manera ágil. Además, TensorFlow permite exportar los modelos de inteligencia artificial a un formato conocido como TFLite (o TensorFlow Lite), especialmente optimizado para dispositivos de bajos recursos como son los móviles.

5.5.2. Diseño de la arquitectura tecnológica para el desarrollo frontend/ backend de la aplicación móvil

Para desarrollar el backend y el frontend de la aplicación móvil se utilizará el lenguaje de Kotlin, ya que es el lenguaje que Google recomienda para el desarrollo en Android. Además, al ser un lenguaje desarrollado por Google, facilita la integración con la librería de TensorFlow, desarrollada también por Google[39].

5.6. Diseño de la persistencia de datos locales

Una de las principales características que tendrá la aplicación será que el procesamiento completo de los datos se hará de forma completamente local, sin necesidad de servicios externos que puedan poner en compromiso la privacidad de los usuarios. Por este motivo es necesario establecer la forma en la que se almacenarán los datos en el dispositivo, de manera que sean de fácil acceso pero a su vez contengan toda la información necesaria.

Para la persistencia de los datos en el dispositivo móvil se utilizará una librería llamada Room[33]. Room proporciona una capa de abstracción sobre SQLite, un sistema de gestión de bases de datos ligero y ampliamente utilizado en el ámbito del desarrollo de aplicaciones móviles.

Para almacenar las predicciones anteriores se utilizará una tabla llamada History, en la cual se almacenarán los datos con la siguiente estructura:

- **id:** Número identificativo generado por el sistema que permitirá identificar de forma única a cada una de las predicciones.
- **imagePath:** Ruta absoluta donde se almacena la imagen original en el dispositivo.
- **maskPath:** Ruta absoluta donde se almacena la máscara de la imagen.
- **croppedImagePath:** Ruta absoluta donde se almacenará la imagen recortada según su máscara.

- **prediction:** Predicción del modelo de aprendizaje automático. Podrá tener dos valores: benigno, si no se ha detectado que la lesión sea cancerígena o maligno, si se ha detectado que la lesión es cancerígena.
- **confidence:** Nivel de confianza del modelo de aprendizaje automático sobre su predicción.
- **timestamp:** Fecha en la que se ha realizado la predicción.

En cuanto a los modelos de aprendizaje automático, se almacenarán también dentro del propio dispositivo, en formato **.tflite**, lo que permitirá no depender de servicios externos que puedan comprometer los datos médicos del usuario, así como su privacidad.

Capítulo 6

Implementación

Una vez establecidas las pautas de diseño de la aplicación, en este capítulo se describe como se ha realizado la implementación. Para ello, se analiza como se realizó la creación de los modelos de aprendizaje automático, así como su entrenamiento. Posteriormente, se explica como se creó la aplicación y cómo los modelos que se habían diseñado se han introducido en la aplicación.

6.1. Configuración del entorno de desarrollo para la implementación y entrenamiento de modelos de aprendizaje automático

El primer paso para implementar los modelos de aprendizaje automático identificados es el de configurar el entorno en el que se trabajó. En este caso, y como se ha mencionado en el capítulo anterior, nuestro entorno se tratará de Google Colab[5]. Colab nos proporciona todas las herramientas necesarias para el desarrollo de los modelos a implementar, ya que viene preconfigurado con todas las librerías que serán necesarias para el desarrollo, que en este caso son:

- **tensorflow:** Librería esencial para el desarrollo de modelos de aprendizaje automático.
- **pandas:** Librería enfocada en el análisis y manejo de datos. Será de utilidad en la fase de clasificación ya que la información sobre las lesiones que contiene el conjunto de datos con el que se trabajó se encuentra en un fichero con formato csv.
- **kagglehub:** Librería de Kaggle[19] que permitirá descargar de forma sencilla cualquier conjunto de datos de la página web **kaggle.com**
- **pathlib:** Librería para el manejo de archivos que nos facilitará trabajar con grandes conjuntos de imágenes.

6.1. Configuración del entorno de desarrollo para la implementación y entrenamiento de modelos de aprendizaje automático

- **numpy:** Librería especialmente optimizada para realizar operaciones matemáticas y que, además, proporciona estructuras de datos optimizadas y usadas por TensorFlow.
- **matplotlib:** Librería que facilitará realizar gráficas comparativas que permitirán seleccionar el mejor modelo de aprendizaje automático.
- **sklearn:** Librería con una amplia cantidad de herramientas para modelos de aprendizaje automático. En este caso será utilizada para dividir el conjunto de datos en datos de entrenamiento y datos de validación.

Como se ha mencionado anteriormente, Colab ya viene preconfigurado con ciertas librerías, entre las que se encuentran las que se utilizarán, por lo que no es necesario descargar ninguna dependencia adicional. En cuanto al entorno de ejecución, es decir, la máquina física donde se iba a ejecutar nuestro código, se seleccionó una máquina con una unidad de procesamiento gráfico A1000[22]. Esta es la unidad de procesamiento gráfico más potente de la que dispone la plataforma de Google Colab, lo cual permite optimizar el tiempo de entrenamiento y evaluación de los modelos, y agilizar el proceso de desarrollo.

Una vez configurado el entorno de desarrollo, se pasa a describir el desarrollo del modelo de aprendizaje automático. Para ello, se estableció un tamaño estándar que tendrían nuestras imágenes, como se mencionó en el RD-002, de 256x256 píxeles. Este tamaño nos permite tener unas imágenes con una calidad suficiente como para detectar características en ellas a la vez que no muy pesadas, lo que permitirá entrenar el modelo de una forma rápida y eficaz.

Además, se implementaron una serie de funciones para ayudar con el manejo de las imágenes. A continuación, se detallan sus nombres y su funcionalidad:

- **normalize:** Función encargada de normalizar los píxeles de la imagen al rango $[0, 1]$ y de ajustar los valores de la máscara para el correcto entrenamiento del modelo.
- **load_image:** Función que, a partir de la ruta de un fichero, carga tanto la imagen como su máscara, las redimensiona al tamaño estándar de 256x256 y las normaliza.
- **display:** Utilidad que permite visualizar en una misma figura la imagen original, la máscara real y la máscara predicha por el modelo, facilitando así la evaluación visual de los resultados.
- **create_mask:** Función que convierte la salida de la red neuronal, que es una matriz de probabilidades, en una máscara de segmentación final.

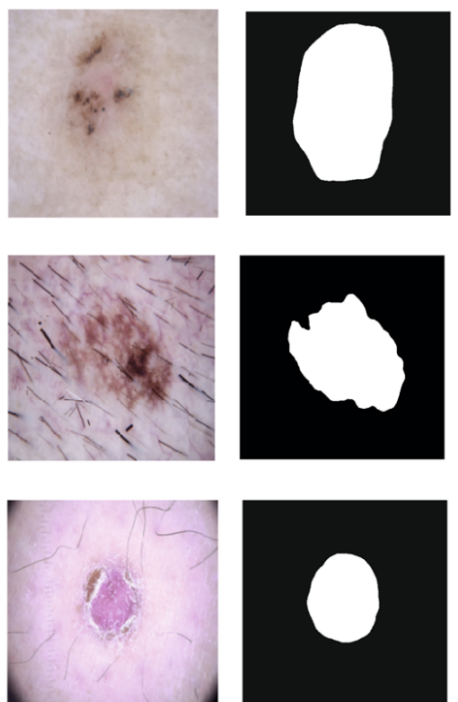


Figura 6.1: Lesiones que se encuentran en el conjunto de datos junto con sus máscaras de segmentación

- **show_predictions:** Función que nos permite realizar una predicción sobre un conjunto de datos y mostrarla de forma visual usando la función *display*.

Para más información sobre la implementación de estas funciones se puede consultar el código que se encuentra disponible en el repositorio de trabajo de Github <https://github.com/pepeilo02/DermaScanner>

En cuanto al conjunto de datos, se seleccionó el mencionado anteriormente, que contiene las máscaras de segmentación (como se puede observar en la Figura 6.1) y que es de gran utilidad para localizar las lesiones encontradas en las imágenes.

Este conjunto de datos, el cual contiene alrededor de 10.000 imágenes, fue dividido en dos subconjuntos, uno de entrenamiento y otro de validación, reservando el 80 % del conjunto para entrenamiento y el resto para la validación, siguiendo las prácticas usadas comúnmente en el desarrollo de modelos de aprendizaje automático[6].

Para la implementación de la red neuronal, en este caso con estructura U-Net[32], se configuró para que recibiera una imagen de 256x256 píxeles. Como salida, la red proporciona un array que contendrá de probabilidad de que el píxel pertenezca o no a la lesión. Para ello, se aplica la estructura

que ya se mencionó en capítulos anteriores (Figura 2.5). En nuestro caso, se aplicaron de forma progresiva cinco capas de codificación, con 8, 16, 32, 64 y 128 filtros de convolución, valiendonos de las herramientas proporcionadas por la librería TensorFlow, haciendo uso de su API Keras.

Para su entrenamiento, se definieron una serie de parámetros que se detallan a continuación:

- **Tamaño de lote {BATCH SIZE}:** En el ámbito del aprendizaje automático, el "batch size" se refiere al tamaño de las subdivisiones que se realizan al conjunto de datos de entrenamiento para procesarlo. El modelo actualiza sus pesos internos después de procesar cada uno de estos lotes. Para este proyecto, se estableció un tamaño de 64.
- **Épocas {EPOCHS}:** Las épocas suponen un ciclo completo de entrenamiento. Es decir, cada vez que se cumple una época, significa que el modelo ha sido entrenado con todos los datos que hay en el conjunto de entrenamiento. En este caso, se han seleccionado 20 épocas, ya que se trata de un número que permite que el modelo aprenda bien las características que le serán de utilidad sin sobreajustarse para adaptarse únicamente a los datos que se encuentran en el conjunto de entrenamiento.
- **Función de pérdida {Loss function}:** La función de pérdida es la métrica utilizada para cuantificar el error del modelo. En esta implementación, se ha enmarcado el problema de la segmentación como una tarea de clasificación multi-clase a nivel de píxel. Para cada píxel de la imagen, el modelo debe decidir si pertenece a la clase "fondo" (clase 0) o a la clase "lesión" (clase 1).

Para la capa de salida de nuestro modelo, se ha elegido la función de activación softmax porque convierte las salidas del modelo en una distribución de probabilidad sobre las dos clases (fondo y lesión). Esto significa que la suma de las probabilidades asignadas a cada clase será 1, lo que permite una interpretación clara de la predicción del modelo.

La función de pérdida seleccionada para trabajar en conjunto con la salida *softmax* es la **entropía cruzada categórica dispersa** (*Sparse Categorical Cross-Entropy*). Esta función es ideal por varias razones:

1. **Eficiencia:** Está diseñada para medir la diferencia entre la distribución de probabilidad predicha por el modelo (salida de softmax) y la distribución de probabilidad real (etiquetas verdaderas).
2. **Simplicidad:** La principal ventaja es que esta función de pérdida no requiere que las etiquetas de clase sean convertidas a un formato *one-hot*. En su lugar, acepta las etiquetas directamente como números enteros (0 para fondo, 1 para lesión), lo que



Figura 6.2: Ejemplo de predicción tras el entrenamiento. De izquierda a derecha, se puede observar la imagen real, la máscara autentica y la máscara predicha por el modelo

simplifica el preprocesamiento de los datos y reduce el consumo de memoria, especialmente en problemas con un gran número de clases.

Su fórmula para un único píxel se define como:

$$L(y, \hat{y}) = -\log(\hat{y}_c)$$

Donde los parámetros son:

- $L(y, \hat{y})$: El valor de la pérdida para un píxel individual.
 - c : Es el índice de la **clase verdadera** para ese píxel (e.g., $c = 0$ si el píxel pertenece al fondo, $c = 1$ si pertenece a la lesión).
 - $\hat{y}_c$: Es la **probabilidad predicha** por el modelo para la clase correcta c , obtenida de la capa de salida con activación softmax[14].
- **Optimizador {Optimizer}**: Es el algoritmo encargado de alterar los pesos del modelo para minimizar la función de pérdida. En este proyecto se ha utilizado la función “Adam”[20] ya que es un estándar en la industria y es ampliamente utilizado para tareas de aprendizaje automático.

Tras definir todos estos parámetros, la siguiente fase sería la del entrenamiento del modelo, que se realizaría de forma automática por medio de las librerías mencionadas anteriormente.

Tras analizar los resultados obtenidos tras el entrenamiento (Figuras 6.2 y 6.3), se puede concluir que el entrenamiento ha sido exitoso, ya que además del éxito a primera vista, se puede observar que la exactitud en las predicciones alcanza un 95 % en la última época de entrenamiento (Figura 6.4).

Con estos resultados se puede concluir que el entrenamiento se ha realizado de forma exitosa y se puede pasar a implementar el segundo modelo, que en este caso se encargará de clasificar las imágenes recortadas en la zona de la lesión.

Para esta segunda fase se implementaron una serie de funciones extras, que se detallan a continuación:

6.1. Configuración del entorno de desarrollo para la implementación y entrenamiento de modelos de aprendizaje automático



Figura 6.3: Otro ejemplo de predicción tras el entrenamiento

```
Sample Prediction after epoch 20
125/125 ————— 68s 547ms/step - accuracy: 0.9596 - loss: 0.1030 - val_accuracy: 0.9568 - val_loss: 0.1124
```

Figura 6.4: Salida del programa tras la última época de entrenamiento. De izquierda a derecha: Tiempo de entrenamiento para esa época, exactitud y pérdida en el conjunto de datos de entrenamiento y en el conjunto de validación

- **crop_image:** Función para que, dada una máscara binaria, devuelva una imagen recortada, con la zona de la máscara y un fondo negro.
- **crop_and_save_dataset:** Función para que, dadas unas imágenes, sus correspondientes máscaras y las etiquetas de clasificación (benigna o maligna), recorte esas imágenes y las guarde en los directorios con el nombre de su etiqueta.

En cuanto al conjunto de datos de entrenamiento, para este nuevo entrenamiento, el proceso llevado a cabo fue el de recortar todas las imágenes del conjunto de datos original con sus respectivas máscaras y crear dos nuevos subconjuntos de entrenamiento y validación.

Para la creación del modelo del aprendizaje profundo encargado de clasificar las lesiones en la piel en malignas o benignas, se utilizó la API de Keras, que se encuentra integrada dentro de la librería de TensorFlow, para descargar el modelo pre-entrenado de MobileNetV3. Esta técnica, que consiste en utilizar un modelo previamente entrenado, se conoce como transferencia de aprendizaje (transfer learning). Su principal ventaja es que permite aprovechar el conocimiento que el modelo ya ha adquirido al ser entrenado con un conjunto de datos masivo y generalista, como es el caso de ImageNet[11].

Los parámetros para el entrenamiento serán los mismos que los mencionados anteriormente para el modelo encargado de la segmentación de las imágenes.

Para llevar a cabo el entrenamiento previo, se llevaron a cabo los siguientes pasos:

1. **Carga del modelo previamente entrenado:** El primer paso fue el de cargar el modelo previamente entrenado.

2. Congelación de las capas del modelo previamente entrenado:

El siguiente paso consiste en “congelar” las capas del modelo. Esto significa que las capas de neuronas artificiales que pertenezcan al modelo base no cambiarán sus pesos en la fase de entrenamiento.

3. Adición de un nuevo clasificador:

Sobre la base ya definida, se definieron unas capas extras, las cuales sí fueron entrenadas. La última capa tiene dos salidas, la probabilidad de que la lesión sea benigna y la probabilidad de que sea maligna.

El modelo clasificador únicamente recibe como entrada una imagen. Esto se debe a que otros parámetros como la edad del paciente, su sexo, etc. podrían ser de interés pero no se poseen los conocimientos médicos suficientes para identificar qué tipo de parámetros podrían ser determinantes a la hora de diagnosticar una lesión en la piel como maligna y cuáles únicamente añadirían ruido a los modelos, llegando a tener consecuencias como bajar la eficacia del modelo o que el modelo tuviese en cuenta factores que no son determinantes para la detección de melanomas. Por estas razones se ha optado por utilizar únicamente imágenes como parámetro para la detección de melanomas.

El proceso de entrenamiento es similar al del primer modelo y se realiza también de forma automática. Tras este, se obtuvieron unos resultados que eran satisfactorios: 94 % de exactitud en las predicciones respecto al conjunto de datos de entrenamiento y un 88 % en el conjunto de datos de validación. Aunque no sea tan elevado como en el anterior modelo, hay que tener en cuenta que es una aplicación que servirá, como se ha mencionado en la introducción, como apoyo para la detección, y el estado de las lesiones siempre será corroborado por un experto, por lo que estas cifras son suficientes para crear una aplicación eficaz. Además, se pudieron corroborar las predicciones de forma manual visualizándolas en un gráfico (Figura 6.5 y 6.6)

Tras esto, se exportaron los modelos en formato **.tflite** para su posterior uso en la aplicación.

6.2. Configuración del entorno para el desarrollo de la aplicación móvil.

Para el desarrollo de la aplicación móvil, se optó por una implementación nativa para la plataforma Android, utilizando las herramientas y lenguajes recomendados por Google. El entorno de desarrollo principal fue Android Studio, el IDE oficial para la creación de aplicaciones Android.

Como ya se ha mencionado anteriormente, el lenguaje utilizado se trata de Kotlin, un lenguaje que se integra a la perfección con Android Studio.

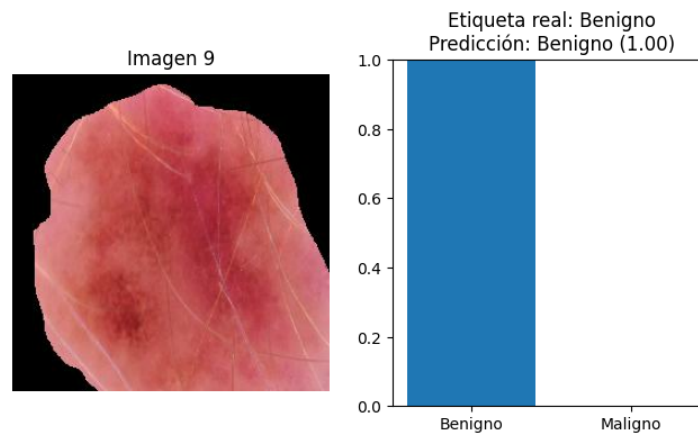


Figura 6.5: Predicción del modelo clasificador

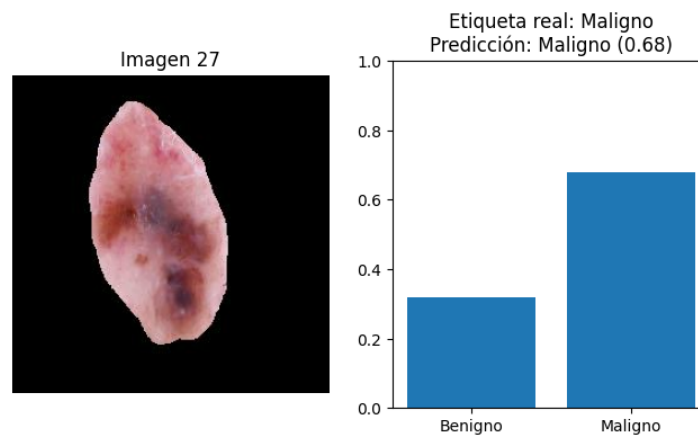


Figura 6.6: Otra predicción del modelo clasificador, en este caso predice maligna

Para la implementación de las funcionalidades clave, se integraron las siguientes librerías de Android Jetpack y de terceros, gestionadas a través del sistema de dependencias Gradle:

- **TensorFlow Lite Android Support Library:** Este conjunto de librerías proporcionan la capacidad de ejecutar modelos tflite en dispositivos móviles de una forma sencilla y optimizada.
- **Jetpack Compose:** Esta librería es de utilidad para diseñar las pantallas e interfaces de la aplicación de una forma fácil y dinámica.
- **CameraX:** Se empleó la librería CameraX de Jetpack para gestionar el acceso a la cámara del dispositivo. Esta API, compatible con el

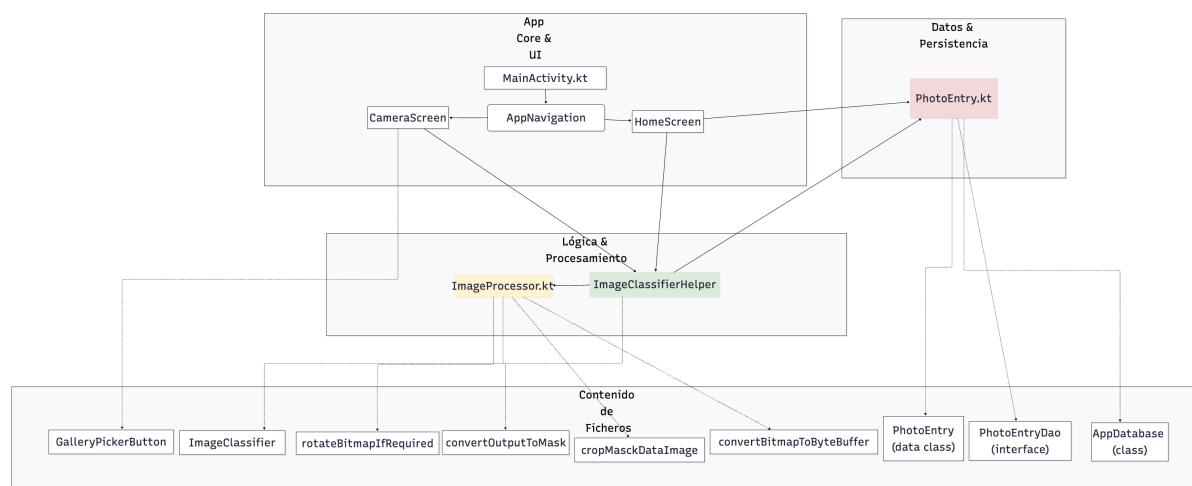


Figura 6.7: Diagrama de la arquitectura de la aplicación

98 % de dispositivos móviles actualmente aporta una forma sencilla de acceder a la cámara del dispositivo, así como tomar fotografías con el mismo.

- **Activity Result APIs (Photo Picker):** Para la selección de imágenes desde la galería, se utilizó el selector de fotos de Android, implementado a través de las APIs de resultado de actividad modernas, lo que garantiza una experiencia de usuario segura y estandarizada.
- **Room Persistence Library:** Para la gestión del historial de predicciones, se implementó una base de datos local utilizando Room, la capa de abstracción sobre SQLite recomendada en Android.

6.3. Diagrama de arquitectura

En la figura 6.7 podemos observar un diagrama que representa la arquitectura del sistema. En el se observan tres partes diferenciadas:

- **App Core & UI:** Este es el núcleo de la aplicación y será la parte encargada de almacenar y gestionar las pantallas que contengan el historial, las predicciones y la pantalla de la cámara.
- **Logica y procesamiento:** En esta parte se encuentran las funciones encargadas de gestionar todo el procesamiento de la imagen (redimensión, normalización, etc.) además de la función encargada de la ejecución de modelos de aprendizaje automático.

- **Datos y persistencia:** Aquí se encuentran las funciones encargadas de gestionar la persistencia de las predicciones, por medio del patrón DAO[24].

6.4. Desarrollo de la aplicación y lógica de negocio

En este apartado se detalla cómo se implementaron los componentes de la aplicación mostrados en el diagrama de arquitectura (Figura 6.7), siguiendo las pautas de diseño del capítulo anterior y utilizando las herramientas y librerías ya mencionadas.

6.4.1. Persistencia de datos con Room

Para cumplir con el requisito de almacenar el historial de análisis en el dispositivo del usuario, se implementó una base de datos local utilizando la librería Room. La estructura de datos se definió en el fichero PhotoEntry.kt:

- Se creó la entidad PhotoEntry, una clase de datos que representa la tabla en la base de datos y almacena toda la información relevante de un análisis: las rutas a las imágenes, la predicción, el nivel de confianza y la fecha.
- Se definió la interfaz PhotoEntryDao, que abstrae el acceso a los datos siguiendo el patrón DAO (Data Access Object) [24]. Esta interfaz contiene los métodos para realizar las operaciones CRUD (Crear, Leer, Borrar) sobre las entradas del historial.
- Finalmente, se configuró la clase principal de la base de datos AppDatabase, que une la entidad y el DAO.

6.4.2. Implementación de la interfaz con Jetpack Compose

Toda la interfaz de usuario se construyó de forma declarativa con Jetpack Compose. Esto permitió crear una interfaz reactiva y moderna, cuya lógica se encuentra en el fichero MainActivity.kt.

- **Navegación y pantallas:** La aplicación se estructura en dos pantallas principales, HomeScreen y CameraScreen. La navegación entre ellas se gestiona de forma simple, controlando el estado de la pantalla actual. Ambas pantallas se pueden ver en las figuras 6.8 y 6.9.
- **Pantalla de historial (HomeScreen):** Esta pantalla muestra la lista de análisis previos obtenidos a través del DAO de Room. Cada elemento permite visualizar los detalles de la predicción en un diálogo emergente, incluyendo la imagen original y la recortada, y da la opción de eliminar la entrada.

- **Pantalla de cámara (CameraScreen):** Esta pantalla integra la vista previa de la cámara. Además del botón de captura, incluye un botón para abrir la galería del dispositivo, implementado con la API `PickVisualMedia`. Un recuadro visual ayuda al usuario a centrar la lesión, y la lógica implementada en `ImageProcessor.kt` se encarga de recortar la imagen capturada según las coordenadas de este recuadro.

6.5. Implementación del pipeline de inferencia

El núcleo funcional de la aplicación reside en el pipeline de análisis, cuya lógica se encapsula en la función `imageClassifier`. Este proceso sigue los dos pasos definidos en el diseño:

1. **Segmentación de la lesión:** La imagen de entrada, ya sea de la cámara o de la galería, se preprocesa (redimensionado y normalización) y se pasa al modelo U-Net. La salida del modelo se convierte en una máscara de segmentación en blanco y negro.
2. **Clasificación de la lesión:** La máscara generada se utiliza para recortar la región de interés de la imagen original. Esta nueva imagen, que contiene únicamente la lesión sobre un fondo negro, se pasa al modelo `MobileNetV3`. La salida de este segundo modelo proporciona la predicción final (“Benigno” o “Maligno”) y su nivel de confianza.

Finalmente, todos los artefactos generados (imagen original, máscara, imagen recortada y resultados del análisis) se empaquetan en un único objeto y se guardan en la base de datos, completando el flujo. El usuario verá por pantalla un dialogo informativo mostrando la predicción, así como la lesión segmentada.

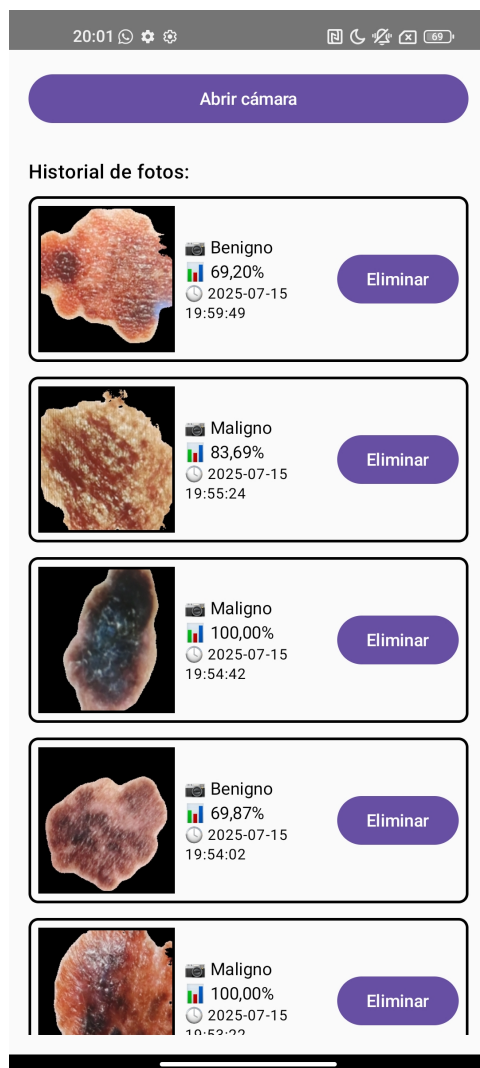


Figura 6.8: Pantalla principal de la aplicación

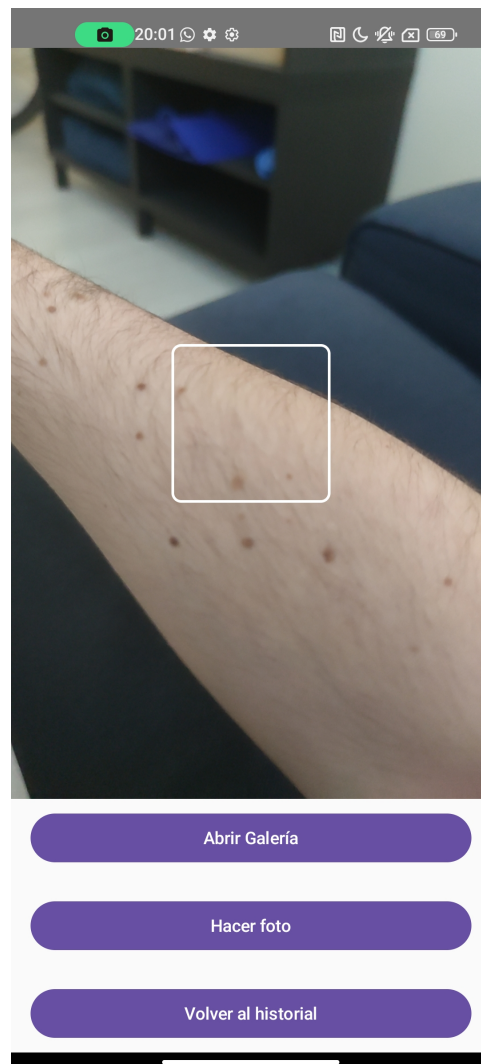


Figura 6.9: Pantalla para tomar imágenes de las lesiones en la piel

Capítulo 7

Evaluación y fase de pruebas

Para la evaluación de los modelos, se obtuvo un conjunto de datos de testeo con el cual se evaluó el desempeño de los modelos frente a datos no utilizados en el entrenamiento. Las métricas utilizadas para la evaluación del modelo de segmentación fueron:

- Exactitud (*Accuracy* en inglés): Métrica que divide el conjunto de predicciones correctas entre el total de predicciones. Sirve para conocer el porcentaje de predicciones acertadas del modelo.
- Precisión (*Precision* en inglés): Métrica que divide el número de predicciones correctas de una clase entre el total de predicciones de esa clase. Sirve para evaluar como de bien se

En cuanto al modelo de clasificación se añadió una métrica extra, la cual es:

- Recall: Esta métrica divide el número de predicciones correctas de una clase entre el total de muestras que realmente pertenecen a esa clase.

En cuanto a la evaluación de la aplicación, se planteó una prueba de campo simulada en la cual se obtuvieron una serie de imágenes cuyas etiquetas ya se conocían y se comprobó la eficacia de la aplicación al tomar fotografías de estas lesiones. Se realizó este tipo de prueba porque la obtención de imágenes de melanomas tomadas por medio de dispositivos móviles es altamente complicado, y con esta prueba de campo simulada se pudo comprobar que el modelo es robusto frente a la menor calidad de la que disponen las cámaras telefónicas comparada con la calidad de las imágenes con las que se ha entrenado el modelo.

7.1. Evaluación individual de los modelos

En cuanto a la evaluación de los modelos, como ya se ha mencionado anteriormente se utilizarán dos métricas y un conjunto de datos que no han sido

utilizados para el entrenamiento de los modelos. A continuación se presentan los resultados obtenidos en las tablas 7.1 y 7.2. Como se puede observar, la mayoría de métricas tienen valores cercanos al 90 %, demostrando que el modelo tiene una gran robustez frente a distintos métodos de evaluación, lo que prueba que se ha construido un modelo sólido y que generaliza de forma correcta.

Tabla 7.1: Resultados de evaluación del modelo de segmentación (U-Net)

Métrica	Valor
Exactitud (Accuracy) a nivel de píxel	95.21 %
Precisión (Precision) a nivel de píxel	89.55 %

Tabla 7.2: Resultados de evaluación del modelo de clasificación (MobileNetV3)

Clase	Precisión	Recall	Nº Muestras
Benigno	92 %	95 %	150
Maligno	80 %	68 %	50
Exactitud (Accuracy)			88 %
Promedio Ponderado	89 %	88.25 %	200

7.2. Evaluación de la aplicación en una prueba de campo simulada

A continuación, se muestra una tabla con información sobre la prueba de campo llevada a cabo. En cada fila se muestra un identificador de imagen, la predicción que realizó la aplicación, el método de captura utilizado (captura de la lesión utilizando la función de cámara de la aplicación o subida directamente desde la galería del dispositivo) y el diagnóstico que realizó la aplicación. Para esta prueba de campo simulada, se seleccionaron 20 imágenes del conjunto de datos BCN20000[10]. De esas 20 imágenes, 10 serán malignas y 10 benignas. Para cada una de las clases, 5 imágenes fueron tomadas desde la cámara disponible en la aplicación y 5 subidas desde la propia galería del dispositivo. Las capturas con las predicciones pueden ser consultadas en el apéndice A

Cabe destacar que esta prueba de campo no está diseñada para probar la eficacia del modelo, puesto que ya se ha testeado anteriormente, sino comprobar que, utilizando ambos modos de enviar imágenes al modelo (desde

la cámara del dispositivo o desde la galería del mismo), recibe las imágenes correctamente y es capaz de detectar lesiones.

En la tabla 7.3 se muestran los resultados de las pruebas de campo. Como se puede observar, en la mayoría de casos la lesión predicha por el modelo y la etiqueta real coinciden, lo que demuestra que el modelo es capaz de identificar lesiones tanto en imágenes que se reciben por medio de la cámara del dispositivo como en aquellas que se suben directamente desde la galería del mismo. Esta prueba demuestra que la aplicación realiza un preprocesamiento correcto de las imágenes recibidas por ambos métodos de subida.

Tabla 7.3: Resultados de la prueba de campo simulada con el conjunto BCN20000.

ID Prueba	Diagnóstico Real	Método de Captura	Diagnóstico de la App	Confianza	Resultado
B-CAM-1	Benigno	Cámara	Benigno	99.99 %	Correcto
B-CAM-2	Benigno	Cámara	Benigno	99.99 %	Correcto
B-CAM-3	Benigno	Cámara	Benigno	99.91 %	Correcto
B-CAM-4	Benigno	Cámara	Benigno	100 %	Correcto
B-CAM-5	Benigno	Cámara	Benigno	99.34 %	Correcto
B-GAL-1	Benigno	Galería	Benigno	99.88 %	Correcto
B-GAL-2	Benigno	Galería	Benigno	93.32 %	Correcto
B-GAL-3	Benigno	Galería	Benigno	71.66 %	Correcto
B-GAL-4	Benigno	Galería	Maligno	94.93 %	Incorrecto
B-GAL-5	Benigno	Galería	Benigno	98.85 %	Correcto
M-GAL-1	Maligno	Galería	Maligno	82.72 %	Correcto
M-GAL-2	Maligno	Galería	Maligno	99.88 %	Correcto
M-GAL-3	Maligno	Galería	Benigno	91.15 %	Incorrecto
M-GAL-4	Maligno	Galería	Maligno	50.25 %	Correcto
M-GAL-5	Maligno	Galería	Maligno	100 %	Correcto
M-CAM-01	Maligno	Cámara	Maligno	64.84 %	Correcto
M-CAM-02	Maligno	Cámara	Maligno	99.90 %	Correcto
M-CAM-01	Maligno	Cámara	Benigno	58.53 %	Incorrecto
M-CAM-02	Maligno	Cámara	Maligno	99.95 %	Correcto

Continúa en la siguiente página...

– continuación de la Tabla 7.3–

ID Prueba	Diagnóstico Real	Método de Captura	Diagnóstico de la App	Confianza	Resultado
M-CAM-02	Maligno	Cámara	Maligno	90.26 %	Correcto

Capítulo 8

Conclusiones y trabajos futuros

Durante el desarrollo de este proyecto, se ha diseñado una aplicación capaz de analizar fotografías de lesiones en la piel y determinar si se trata de lesiones malignas o benignas. Aunque las predicciones emitidas por la aplicación nunca pueden considerarse definitivas sin la posterior supervisión de un experto, este proyecto ha permitido desarrollar una forma sencilla de analizar las lesiones en la piel de los usuarios permitiéndoles mantener un control exhaustivo sobre las mismas, además de alertarles de aquellas que tengan mayores indicios de convertirse en cancerígenas.

El desarrollo de este proyecto se ha apoyado en las siguientes técnicas y tecnologías:

- **Análisis y preprocesamiento de datos:** Durante la primera fase de este proyecto, se han analizado diversos conjuntos de datos, escogiendo el que se consideraba más adecuado para el objetivo a cumplir. Además, el conjunto de datos se ha adaptado y normalizado para poder cumplir con los estándares que se fijaron.
- **Desarrollo de modelos de aprendizaje automático:** Se han analizado distintas arquitecturas de modelos de aprendizaje automático, comparando las ventajas que ofrecían unas sobre otras e implementando la escogida. Además, se han utilizado diferentes métricas de valoración para comprobar que los resultados obtenidos tras la implementación y el entrenamiento eran satisfactorios.
- **Uso de técnicas avanzadas de aprendizaje automático:** Además de implementar las arquitecturas desde cero, se han utilizado modelos previamente entrenados, aprovechando sus conocimientos previos mediante la técnica de fine-tuning, que consiste en utilizar un modelo ya existente y entrenarlo para que realice con precisión una tarea en concreto, en este caso la clasificación de lesiones en la piel. Esto ha

permitido reducir los tiempos de entrenamiento, agilizando el proceso de implementación.

- **Cuantización de modelos para el despliegue en dispositivos móviles:** Se han aprovechado las APIs de distintas librerías de aprendizaje automático ampliamente utilizadas para reducir el tamaño de los modelos y permitir su ejecución desde dispositivos con menores capacidades computacionales como son los dispositivos móviles.
- **Implementación móvil:** Se ha implementado una interfaz gráfica sencilla, que permite una navegación sencilla entre pantallas y aporta una solución integrada para tomar capturas de las lesiones desde la propia aplicación por medio de las APIs que proporciona el propio sistema operativo.

Todo esto ha permitido desarrollar una solución sencilla y accesible para un gran número de usuarios, proporcionándoles una herramienta para detectar de forma temprana posibles lesiones cancerígenas en la piel.

En el ámbito personal, este proyecto me ha ayudado a aprender las fases que componen el desarrollo de modelos de aprendizaje automático, la dificultad de crear modelos que generalicen de una manera correcta y las implicaciones éticas que conlleva el desarrollo de aplicaciones en un ámbito tan delicado como es la salud. Además, me ha hecho comprender la inmensa capacidad que tiene la inteligencia artificial de transformar nuestras vidas a lo largo de los próximos años en diferentes campos, sirviendo no como un sustituto sino como un apoyo para potenciar el avance humano en las siguientes décadas.

Como ya se ha mencionado anteriormente, el campo de la inteligencia artificial y el aprendizaje automático es un campo en constante evolución, con la aparición de nuevos avances casi a diario, haciendo que la solución implementada en este proyecto pueda quedar obsoleta en un breve periodo de tiempo. Es por eso que en un futuro se buscará implementar todos estos avances en el proyecto, permitiendo obtener cada vez diagnósticos más precisos.

En concreto, una de las tecnologías que se buscará implementar en un futuro será la integración de modelos de lenguaje multimodales dentro de nuestra aplicación. Un modelo de lenguaje es un tipo de modelo que permite analizar entradas de texto y otorgar respuestas en lenguaje natural, con un tono y un estilo propios de una persona. En concreto, los modelos multimodales pueden también recibir como entrada distintos formatos multimedia, como pueden ser fotos o videos. Esto es de gran interés para **trabajos futuros**, ya que permitiría no solo aportar una predicción y un porcentaje de confianza, sino que el usuario también sería capaz de recibir una explicación en lenguaje natural de los motivos que han llevado al modelo a determinar una predicción. Esta nueva implementación favorecería las tendencias

actuales en el campo de la inteligencia artificial, en las cuales se busca no solo que la inteligencia artificial otorgue una respuesta sino que aporte argumentos para justificarla, haciendo que sea más confiable para el usuario y que no sea vista como una caja negra que simplemente otorga respuestas sin justificación.

Es por eso que este proyecto se trata de un proyecto en constante evolución, que avanzará al mismo tiempo que lo hacen los modelos y técnicas de aprendizaje profundo.

Capítulo 9

Bibliografía

- [1] Charu C. Aggarwal. *Neural Networks and Deep Learning. A Textbook*. Cham: Springer, 2018, pág. 497. ISBN: 978-3-319-94462-3. DOI: 10 . 1007/978-3-319-94463-0.
- [2] Akhil Agnihotri, Prathamesh Saraf y Kriti Rajesh Bapnad. “A convolutional neural network approach towards self-driving cars”. En: *2019 IEEE 16th India Council International Conference (INDICON)*. IEEE. 2019, págs. 1-4.
- [3] Apple Inc. *About Face ID advanced technology*. 2025. URL: <https://support.apple.com/en-us/102381>.
- [4] Dor Bank, Noam Koenigstein y Raja Giryes. *Autoencoders*. 2021. arXiv: 2003.05991 [cs.LG]. URL: <https://arxiv.org/abs/2003.05991>.
- [5] Ekaba Bisong. *Building Machine Learning and Deep Learning Models on Google Cloud Platform: A Comprehensive Guide for Beginners*. Ene. de 2019. ISBN: 978-1-4842-4469-2. DOI: 10 . 1007/978-1-4842-4470-8.
- [6] Gavin Cawley y Nicola Talbot. “On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation”. En: *Journal of Machine Learning Research* 11 (jul. de 2010), págs. 2079-2107.
- [7] François Chollet. *keras*. <https://github.com/fchollet/keras>. 2015.
- [8] KR1442 Chowdhary y KR Chowdhary. “Natural language processing”. En: *Fundamentals of artificial intelligence* (2020), págs. 603-649.
- [9] Noel Codella et al. “Skin lesion analysis toward melanoma detection 2018: A challenge hosted by the international skin imaging collaboration (isic)”. En: *arXiv preprint arXiv:1902.03368* (2019).

-
- [10] Marc Combalia et al. *BCN20000: Dermoscopic Lesions in the Wild*. 2019. arXiv: 1908.02288 [eess.IV]. URL: <https://arxiv.org/abs/1908.02288>.
 - [11] Jia Deng et al. “Imagenet: A large-scale hierarchical image database”. En: *2009 IEEE conference on computer vision and pattern recognition*. Ieee. 2009, págs. 248-255.
 - [12] Alexey Dosovitskiy et al. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. 2021. arXiv: 2010.11929 [cs.CV]. URL: <https://arxiv.org/abs/2010.11929>.
 - [13] Ian J. Goodfellow et al. *Generative Adversarial Networks*. 2014. arXiv: 1406.2661 [stat.ML]. URL: <https://arxiv.org/abs/1406.2661>.
 - [14] Jiuxiang Gu et al. *Exploring the Frontiers of Softmax: Provable Optimization, Applications in Diffusion Model, and Beyond*. 2024. arXiv: 2405.03251 [cs.LG]. URL: <https://arxiv.org/abs/2405.03251>.
 - [15] Kaiming He et al. *Deep Residual Learning for Image Recognition*. 2015. arXiv: 1512.03385 [cs.CV]. URL: <https://arxiv.org/abs/1512.03385>.
 - [16] Andrew Howard et al. “Searching for mobilenetv3”. En: *Proceedings of the IEEE/CVF international conference on computer vision*. 2019, págs. 1314-1324.
 - [17] Andrew G. Howard et al. *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*. 2017. arXiv: 1704.04861 [cs.CV]. URL: <https://arxiv.org/abs/1704.04861>.
 - [18] International Skin Imaging Collaboration (ISIC). *ISIC Mission*. 2025. URL: <https://www.isic-archive.com/mission>.
 - [19] Kaggle. *Kaggle*. 2010. URL: <https://www.kaggle.com/>.
 - [20] Diederik P. Kingma y Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2017. arXiv: 1412.6980 [cs.LG]. URL: <https://arxiv.org/abs/1412.6980>.
 - [21] Economía de Mallorca. *El mercado de la IA crecerá un 35 % por año hasta 2026*. 2024. URL: <https://www.economiademallorca.com/articulo/innovacion/mercado-ia-crecera-35-ano-2026/20240415203454093205.html>.
 - [22] NVIDIA. *NVIDIA RTX A1000 Laptop GPU*. <https://marketplace.nvidia.com/en-us/enterprise/laptops-workstations/nvidia-rtx-a1000/>. 2022.
 - [23] Keiron O’Shea y Ryan Nash. “An Introduction to Convolutional Neural Networks”. En: *CoRR* abs/1511.08458 (2015). arXiv: 1511.08458. URL: <http://arxiv.org/abs/1511.08458>.

- [24] Oracle. *Core J2EE Patterns - Data Access Object*. <https://www.oracle.com/java/technologies/data-access-object.html>. 2025.
- [25] John Øvretveit et al. “Improving quality through effective implementation of information technology in healthcare”. En: *International Journal for Quality in Health Care* 19.5 (ago. de 2007), págs. 259-266. ISSN: 1353-4505. DOI: 10.1093/intqhc/mzm031. eprint: <https://academic.oup.com/intqhc/article-pdf/19/5/259/5094749/mzm031.pdf>. URL: <https://doi.org/10.1093/intqhc/mzm031>.
- [26] Adam Paszke et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. En: *CoRR* abs/1912.01703 (2019). arXiv: 1912.01703. URL: <http://arxiv.org/abs/1912.01703>.
- [27] F. Pedregosa et al. “Scikit-learn: Machine Learning in Python”. En: *Journal of Machine Learning Research* 12 (2011), págs. 2825-2830.
- [28] Project Jupyter. *Jupyter Notebook*. <https://jupyter.org/>. 2025.
- [29] Pranav Rajpurkar et al. “AI in health and medicine”. En: *Nature medicine* 28.1 (2022), págs. 31-38.
- [30] Thomas Riehle. *OOPSLA-1998*. <https://riehle.org/computer-science/research/1998/oopsla-1998.html>. 1998.
- [31] A. Rodrigo Schwartz, C. Gustavo Vial y J. Ricardo Schwartz. “Estrategias de detección precoz de melanoma cutáneo”. En: *Revista Médica Clínica Las Condes* 22.4 (2011). Tema central: Cáncer. prevención y diagnóstico precoz, págs. 466-475. ISSN: 0716-8640. DOI: [https://doi.org/10.1016/S0716-8640\(11\)70452-7](https://doi.org/10.1016/S0716-8640(11)70452-7). URL: <https://www.sciencedirect.com/science/article/pii/S0716864011704527>.
- [32] Olaf Ronneberger, Philipp Fischer y Thomas Brox. “U-Net: Convolutional Networks for Biomedical Image Segmentation”. En: *CoRR* abs/1505.04597 (2015). arXiv: 1505.04597. URL: <http://arxiv.org/abs/1505.04597>.
- [33] *Room release notes*. <https://developer.android.com/jetpack/androidx/releases/room>. 2025.
- [34] Jiacheng Ruan et al. *MALUNet: A Multi-Attention and Light-weight UNet for Skin Lesion Segmentation*. 2022. arXiv: 2211.01784 [eess.IV]. URL: <https://arxiv.org/abs/2211.01784>.
- [35] Olga Russakovsky et al. “ImageNet Large Scale Visual Recognition Challenge”. En: *International Journal of Computer Vision (IJCV)* 115.3 (2015), págs. 211-252. DOI: 10.1007/s11263-015-0816-y.
- [36] Xingjian Shi et al. *Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting*. 2015. arXiv: 1506.04214 [cs.CV]. URL: <https://arxiv.org/abs/1506.04214>.

- [37] Sociedad Española de Oncología Médica. *Se incrementa la incidencia interanual de melanoma en España con 7.881 casos nuevos en 2024*. 2024. URL: <https://seom.org/otros-servicios/noticias/210543-se-incrementa-la-incidencia-interanual-de-melanoma-en-espana-con-7-881-casos-nuevos-en-2024>.
- [38] American Cancer Society. *¿Que es el cáncer de piel tipo melanoma?* 2019. URL: <https://www.cancer.org/es/cancer/tipos/cancer-de-piel-tipo-melanoma/acerca/que-es-melanoma.html>.
- [39] TensorFlow Team. *TensorFlow*. <https://github.com/tensorflow/tensorflow>. 2024.
- [40] Philipp Tschandl, Cliff Rosendahl y Harald Kittler. “The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions”. En: *Scientific Data* 5.1 (ago. de 2018), pág. 180161. ISSN: 2052-4463. DOI: 10.1038/sdata.2018.161. URL: <https://doi.org/10.1038/sdata.2018.161>.
- [41] Philipp Tschandl et al. “Human–computer collaboration for skin cancer recognition”. En: *Nature Medicine* 26.8 (ago. de 2020), págs. 1229-1234. ISSN: 1546-170X. DOI: 10.1038/s41591-020-0942-0. URL: <https://doi.org/10.1038/s41591-020-0942-0>.
- [42] Guido Van Rossum y Fred L Drake Jr. *Python reference manual*. Centrum voor Wiskunde en Informatica Amsterdam, 1995.
- [43] Karl Wieggers y Joy Beatty. *Software Requirements. Best Practices*. Microsoft Press, sep. de 2013. URL: <http://repositorioinstitucionaluacm.mx/jspui/handle/123456789/2502>.
- [44] Wikipedia. *Caso de uso* — *Wikipedia, La enciclopedia libre*. 2024. URL: https://es.wikipedia.org/w/index.php?title=Caso_de_uso&oldid=164298129.
- [45] Wikipedia. *Desarrollo en cascada* — *Wikipedia, La enciclopedia libre*. [Internet; descargado 25-diciembre-2024]. 2024. URL: https://es.wikipedia.org/w/index.php?title=Desarrollo_en_cascada&oldid=164306255.
- [46] Wikipedia. *Requisito funcional* — *Wikipedia, La enciclopedia libre*. 2024. URL: https://es.wikipedia.org/w/index.php?title=Requisito_funcional&oldid=162200043.
- [47] Wikipedia. *Requisito no funcional* — *Wikipedia, La enciclopedia libre*. 2024. URL: https://es.wikipedia.org/w/index.php?title=Requisito_no_funcional&oldid=161120218.

Apéndice A

Imágenes tomadas para la prueba de campo

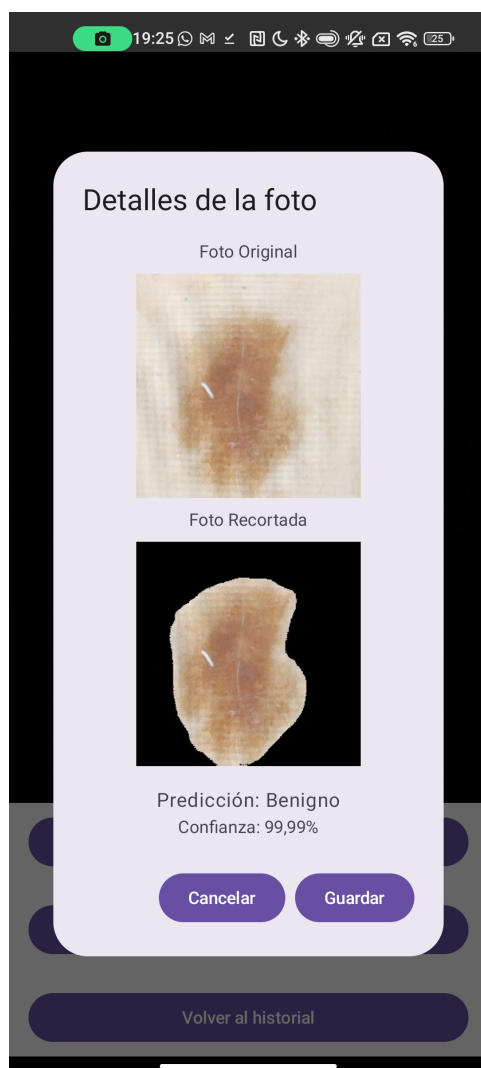


Figura A.1: Predicción B-CAM-1

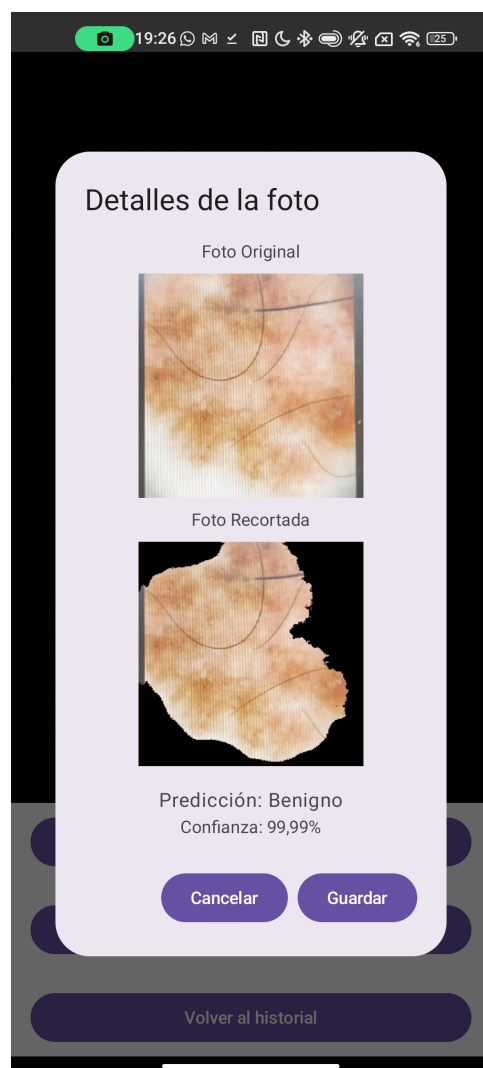


Figura A.2: Predicción B-CAM-2

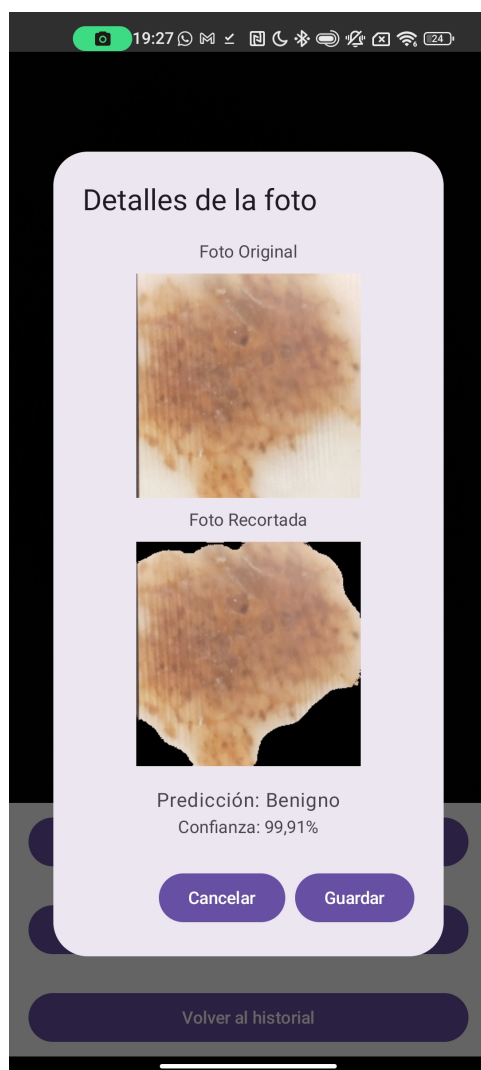


Figura A.3: Predicción B-CAM-3

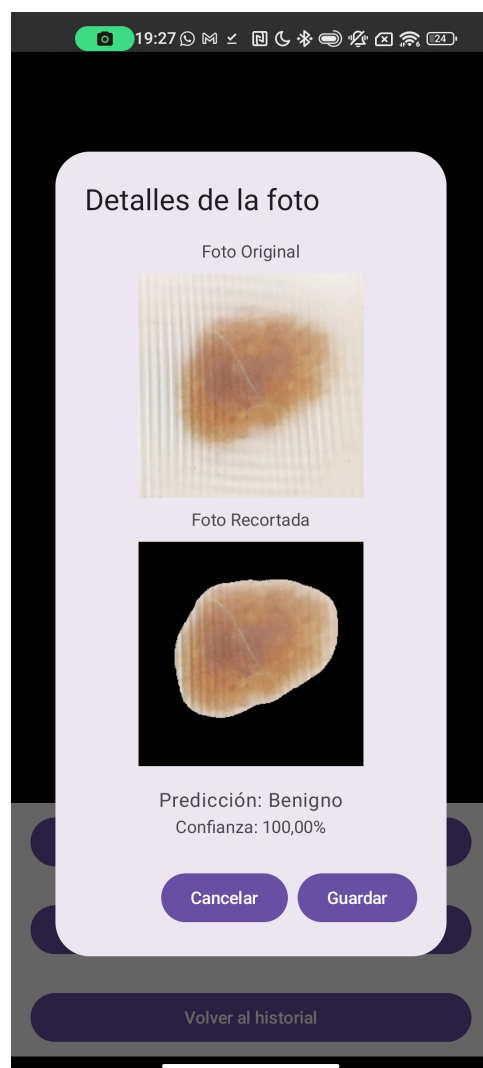


Figura A.4: Predicción B-CAM-4

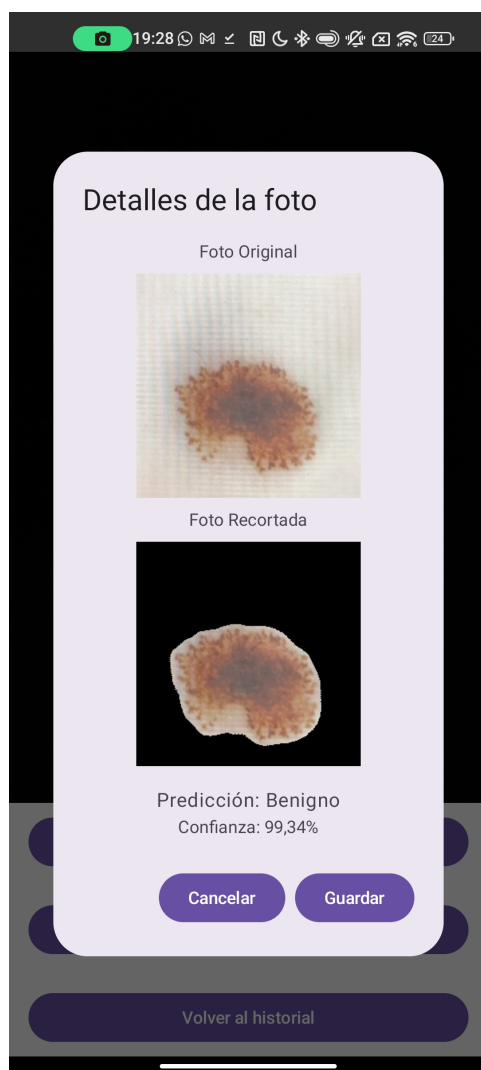


Figura A.5: Predicción B-CAM-5



Figura A.6: Predicción B-GAL-1

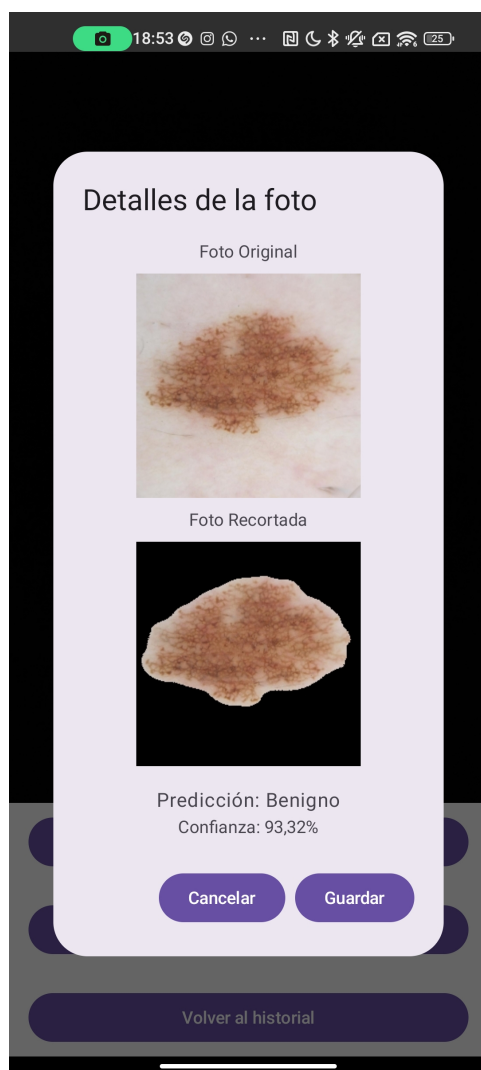


Figura A.7: Predicción B-GAL-2

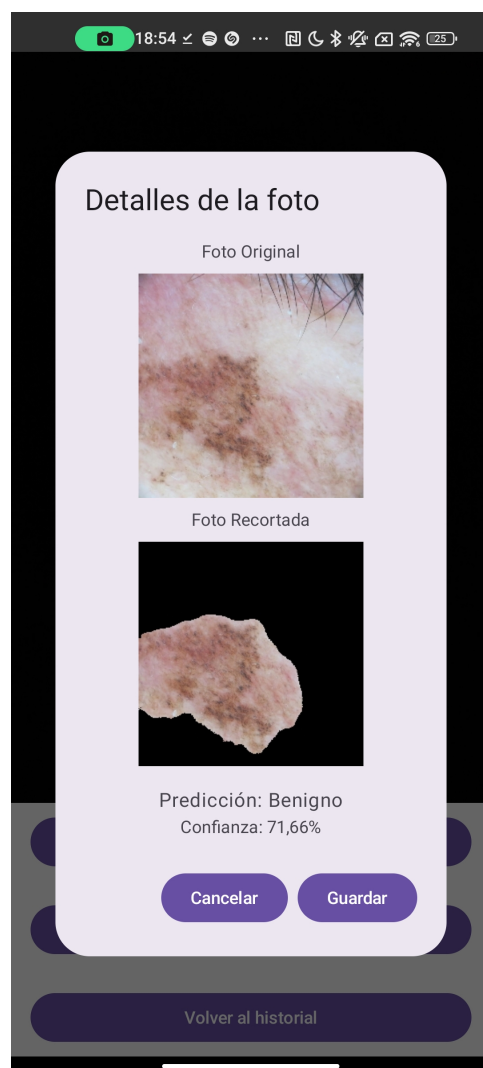


Figura A.8: Predicción B-GAL-3

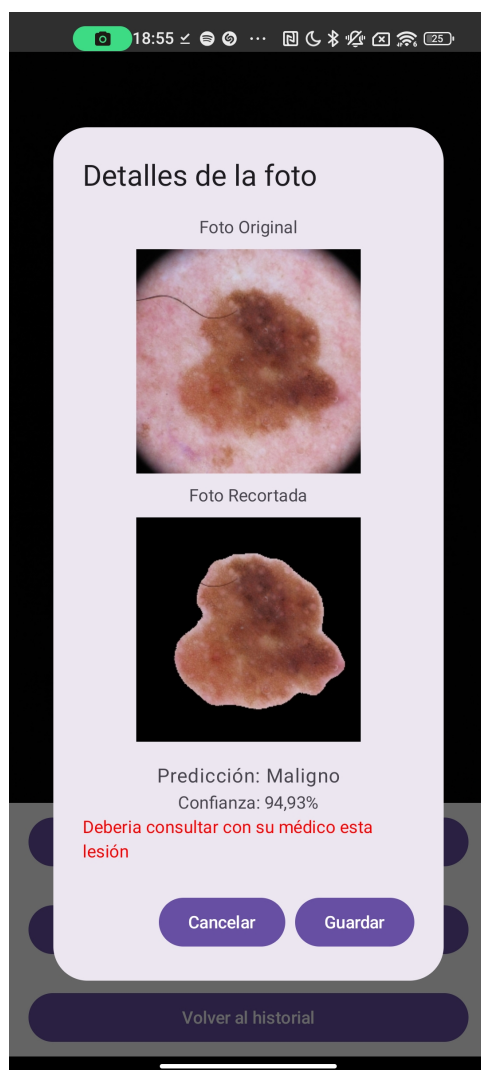


Figura A.9: Predicción B-GAL-4

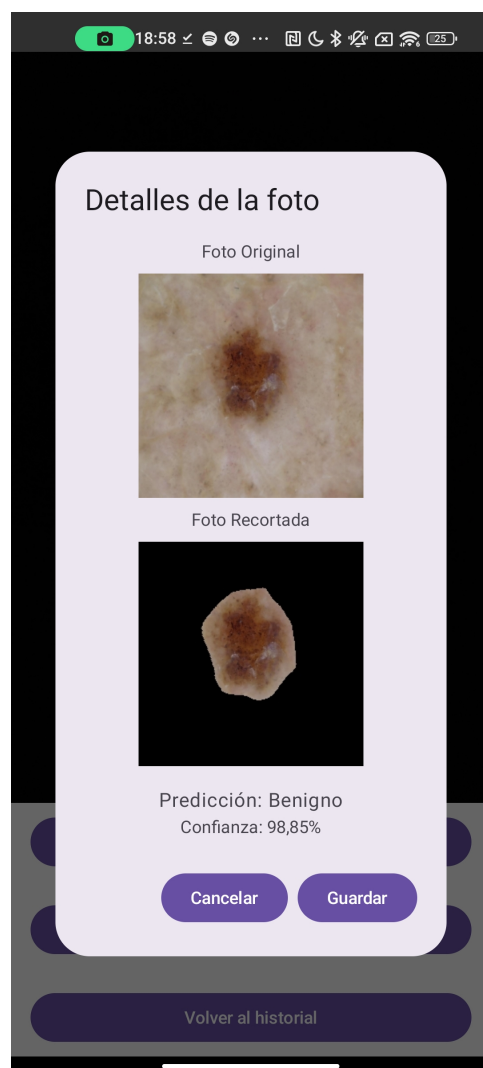


Figura A.10: Predicción B-GAL-5



Figura A.11: Predicción M-GAL-1

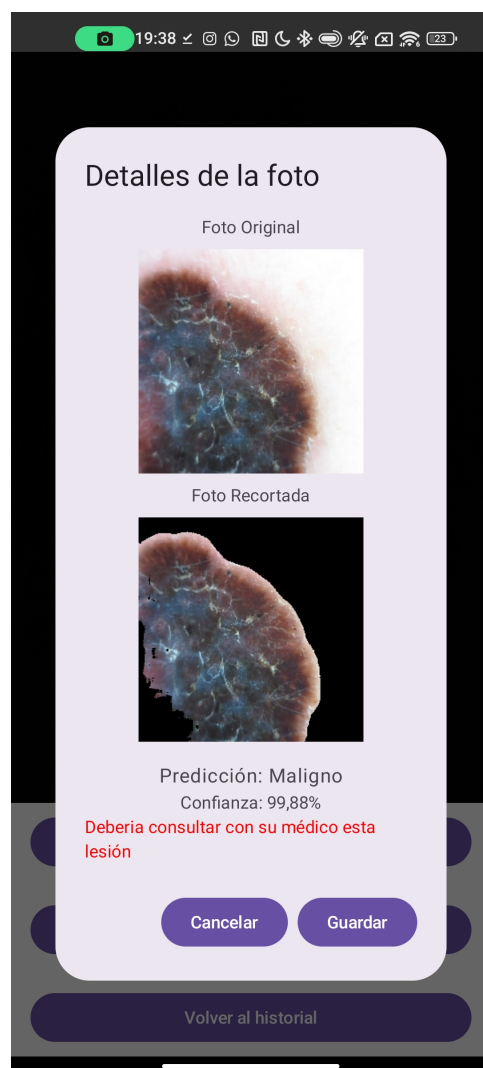


Figura A.12: Predicción M-GAL-2



Figura A.13: Predicción M-GAL-3

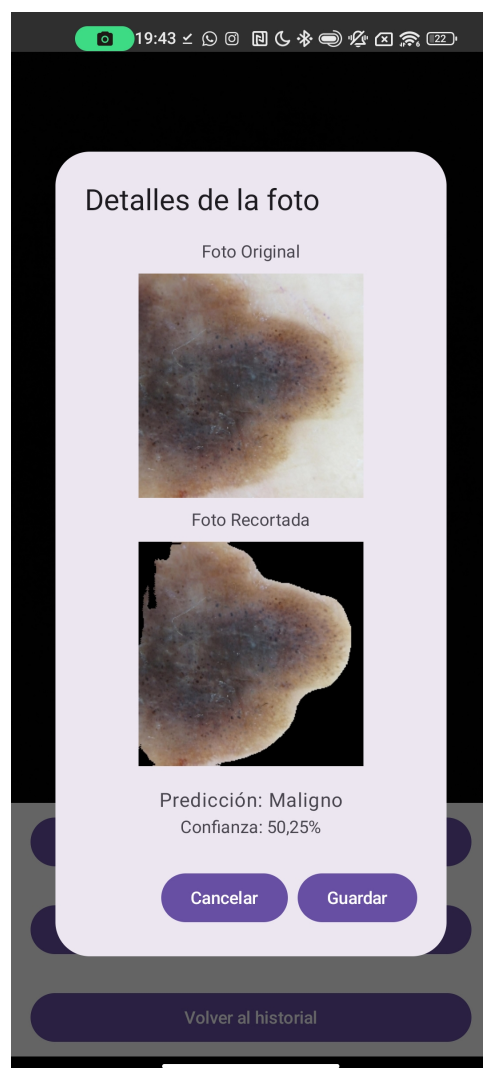


Figura A.14: Predicción M-GAL-4

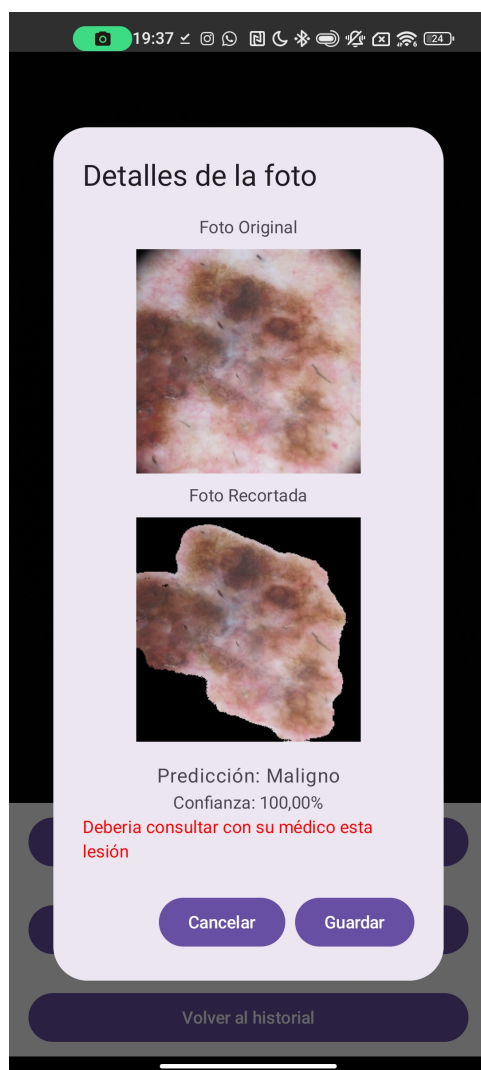


Figura A.15: Predicción M-GAL-5



Figura A.16: Predicción M-CAM-1



Figura A.17: Predicción M-CAM-2

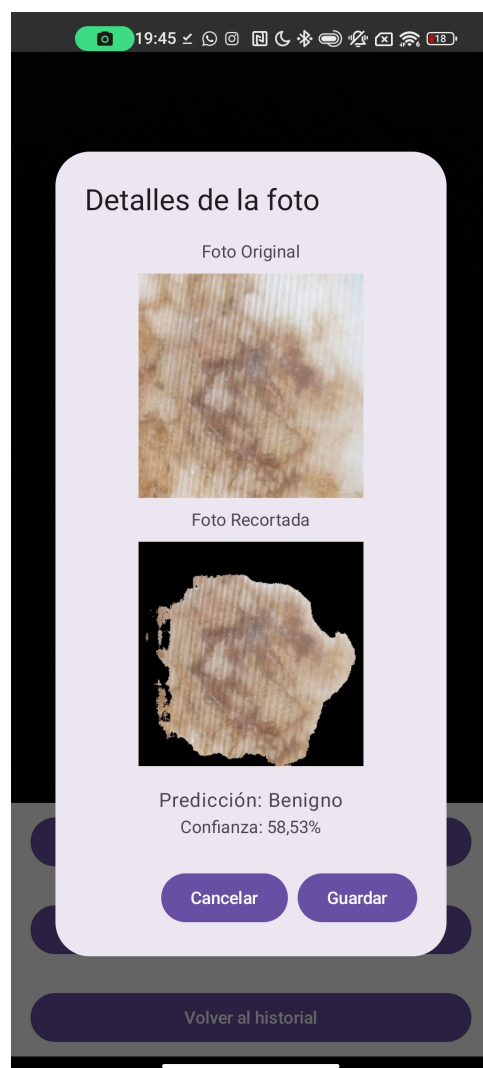


Figura A.18: Predicción M-CAM-3

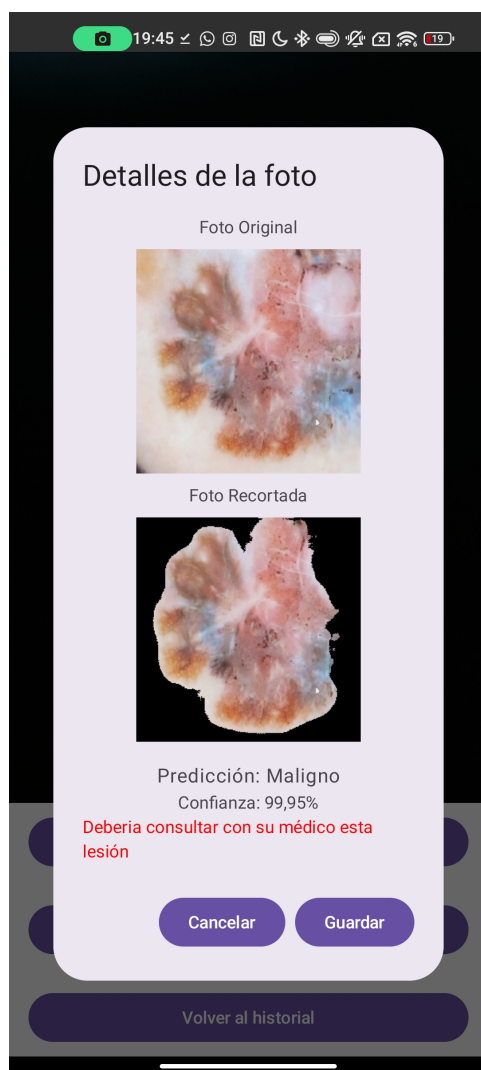


Figura A.19: Predicción M-CAM-4

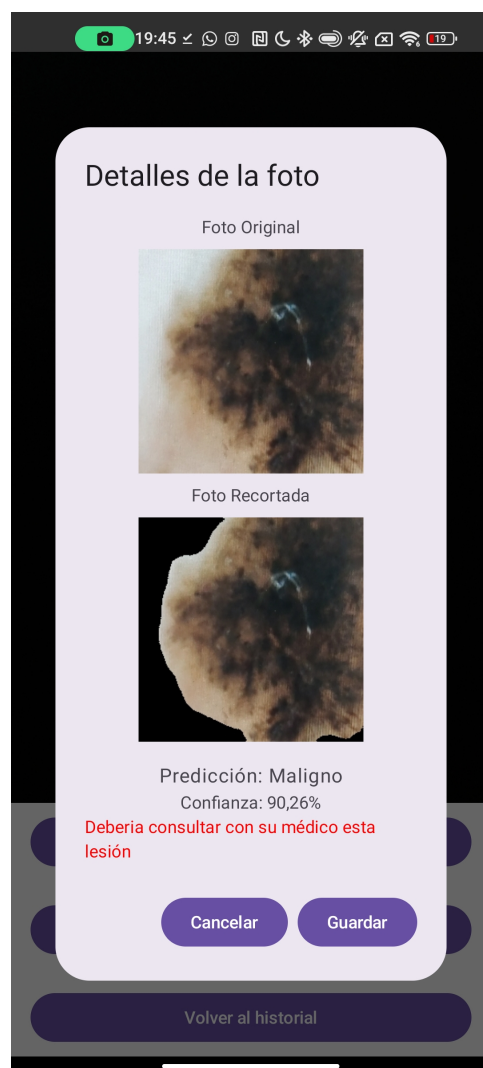


Figura A.20: Predicción M-CAM-5

